ABC ID: 611661470416

AI-POWERED SYSTEM FOR RIPENESS DETECTION AND POST-HARVEST STORAGE ADVICE

A PROJECT REPORT

Submitted to

Jawaharlal Nehru Technological University Kakinada, Kakinada
in partial fulfillment for the award of the degree of
Bachelor of Technology
in
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

Name: K. Suchitra Kamala
Reg No: 22KN1A6145

Under the esteemed guidance of

Dr. P. Rajendra Kumar
Designation,
Department of AI&ML

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
NRI INSTITUTE OF TECHNOLOGY
Autonomous
(Approved by AICTE, Permanently Affiliated to JNTUK, Kakinada)
Accredited by NBA (CSE, ECE, EEE, IT & ME), Accredited by NAAC with 'A' Grade
ISO 9001: 2015 Certified Institution
Pothavarappadu (V), (Via) Nunna, Agiripalli (M), Eluru Dist, PIN: 521212, A.P, India.

2025-26

NRI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

Approved by AICTE, New Delhi: Permanently Affiliated to JNTUK, Kakinada
Accredited by NAAC with "A" GRADE, Accredited by NBA (CSE, ECE, EEE, IT & ME)
An ISO 9001:2015 Certified Institution
Pothavarappadu (V), Agiripalli (M), Eluru District, A.P., India, Pin: 521 212
URL: www.nriit.edu.in, email: principal@nriit.edu.in, Mobile: 8333882444

CERTIFICATE

This is to certify that the Project entitled "AI-POWERED SYSTEM FOR RIPENESS DETECTION AND POST-HARVEST STORAGE ADVICE" is a bonafide work carried out by K. Suchitra Kamala (22KN1A6145) in partial fulfillment for the award of degree of Bachelor of Technology in Artificial Intelligence and Machine Learning of Jawaharlal Nehru Technological University Kakinada, during the year 2022-26.

PROJECT GUIDE | **EXTERNAL EXAMINER** | **HEAD OF THE DEPARTMENT**

INTERNSHIP CERTIFICATE

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that KODALI SUCHITRA KAMALA with Registered No: 22KN1A6145 from Department of Artificial Intelligence and Machine Learning, NRI INSTITUTE OF TECHNOLOGY, Pothavarapadu, Vijayawada. has been successfully completed Short-Term Internship on LangGraph-Based AI Agents with Society for Learning Technologies, Vijayawada in Collaboration with Andhra Pradesh State Council of Higher Education from 03-11-2025 to 15-12-2025.

Internship Coordinator, SOLETE

Secretary, SOLETE

DECLARATION

I hereby declare that the project report titled "AI-POWERED SYSTEM FOR RIPENESS DETECTION AND POST-HARVEST STORAGE ADVICE" is a bonafide work carried out in the Department of Artificial Intelligence and Machine Learning, NRI Institute of Technology, Agiripalli, Vijayawada, during the academic year 2025- 2026, in partial fulfilment for the award of the degree of Bachelor of Technology by JNTU Kakinada. I further declare that this dissertation has not been submitted elsewhere for any Degree.

K. Suchitra Kamala (22KN1A6145)

ACKNOWLEDGEMENT

I take this opportunity to thank all who have rendered their full support to our work. The pleasure, the achievement, the glory, the satisfaction, the reward, the appreciation and the construction of our project cannot be expressed with a few words for their valuable suggestions.

I express my deep sense of gratitude to our Chairman, Dr. R. VENKATA RAO for his constant encouragement and support throughout our academic journey

I thank our Principal, Dr. C. NAGA BHASKAR for providing the necessary infrastructure required for our project.

I extend my sincere and heartfelt thanks to Professor and Head of the Department, Dr. P. RAJENDRA KUMAR for his continuous guidance in completion of our Project work.

I am grateful to our Coordinator, Associate Professor Mrs. S. RAMADEVI for the timely guidance, valuable suggestions, and encouragement during the course of this Project.

I extend my sincere thanks to our Guide, Associate Professor Dr. T. Manasa Veena continuous guidance and support to complete our project successfully.

Finally, I thank the Administrative Officer, Staff Members, Faculty of Department of AI&ML, NRI Institute of Technology, my parents and my friends, directly or indirectly who helped us in the completion of this project.

PROJECT ASSOCIATE
K. Suchitra Kamala
22KN1A6145

Abstract

Based on the situations of the farmers who are facing a major challenge in the agricultural sector for post-harvesting of the fruits and vegetables. This problem majorly occurs due to the improper detection of ripeness and unsuitable storage practices. Traditionally the manual methods to estimate ripeness are often becomes inaccurate, time-consuming, and subjective. To address this To address this issues, this project presents a AI-Powered System which uses to processes images and utilizes machine learning techniques to detect ripeness automatically of a fruit or vegetable and provide post-harvesting storage recommendations to the farmers and the vendors.In previous method they have used YOLOv5, YOLOv6, YOLOv7 and SSD-MobileNetv1 for detecting fruit and classifying their ripeness in real- world images. And used to explain the manual image dataset which consist of strawberries and avocados images under natural conditions with four different stages like- Unripe, Partially Ripe, Ripe, Rotten.

In this dataset the images are naturally captured under the lighting and featured varied backgrounds, occlusions for reflecting real harvesting environments and test robustness and the dataset is bounded by boxes around fruits and corresponding ripeness stage labels the dataset can be available in the Mendeley. The purpose of dataset enables benchmarking of object detection model (YOLOv5, YOLOv6, YOLOv7, and SSD-MobileNetv1) for multi-class fruit ripeness classification, rather than just binary ripe/unripe classification.In previous it have been used MobileNetV2 which is lightweight convolutional neural network designed for efficient feature extraction so based on it in project we are using EfficientNetB0 which is similar to it where as both models aims to achieve high accuracy with reduced computational cost, making them suitable for real-time agricultural image analysis and resource constrained environments. The MobileNetV2 that is used in previous is highly cost-efficient, very low Parameters count(~3-4 million parameters), fast interface time, lower training cost, Uses depthwise separable convolutions, Consumes less GPU/CPU power Shorter training time, Lower accuracy trade-off, Well-suited for mobile and embedded systems, Ideal for large-scale deployment. With the help of MobileNetV2 in the previous method it get overall accuracy of 99.3% and as of now using EfficientNetB0 the project getting the overall accuracy of 99.4%.

INDEX

LIST OF FIGURES

CHAPTER-1
INTRODUCTION

1. Introduction

This project aims to systematically review and compare the efficacy of various machine learning and deep learning methodologies applied to fruit ripeness detection, addressing the critical need for efficient and accurate assessment in agricultural automation (Salim & Mohammed, 2024) (Zhou et al., 2025). Specifically, the focus will be on deep learning approaches that have demonstrated superior accuracy compared to other methods in vision-based systems for fruit ripeness detection (Zhang, 2023; Zhou et al., 2025). Despite these advancements, ongoing challenges persist in achieving high accuracy in complex scenarios involving variable lighting, fruit sizes, and occlusions (Zhang, 2023). This is where the scalability and end-to-end learning capabilities of deep learning models, particularly CNN's, become crucial, allowing them to adapt to diverse datasets and eliminate the need for manual preprocessing and feature engineering (Goh et al., 2024). Moreover, deep learning architectures, such as improved DenseNet, effectively manage challenges like dataset imbalance and sparse data through innovative loss functions and structured sparse operations, optimizing performance in real-world agricultural settings (Vo et al., 2024). Among recent fruit ripeness detection studies, YOLO and its optimized variants such as YOLOv5, YOLOv8, YOLOv9c, CES-YOLO (custom YOLO), and YOLO + Transformer have been extensively utilized due to their real-time detection capability, single-stage object detection, and high accuracy with low inference time (Goh et al., 2024; Salim & Mohammed, 2024; Zhou et al., 2025). Furthermore, their suitability for edge devices and field deployment, coupled with their ability to effectively handle multi-class ripeness stages, makes them highly attractive for precision agriculture applications (Wang et al., 2025; Xiao et al., 2023). Beyond YOLO, CNN-based lightweight models like MobileNet, Lightweight CNN, and Mask R-CNN are also employed, primarily due to their low computational cost and efficiency on mobile and IoT platforms (Singh et al., 2025). Conversely, Transformer-based models, including RT-DETR, Vision Transformer, and Attention-based YOLO, represent an emerging trend, demonstrating

enhanced global context understanding and improved accuracy in complex backgrounds, despite their comparatively higher computational demands (Zhang, 2023).

Among recent fruit ripeness detection studies, YOLO(You Only Look Once) and its optimized variants used like YOLOv5, YOLOv8, YOLOv9c, CES-YOLO(custom YOLO), YOLO + Transformer (Hybrid) due to Real-time detection capability, Single-stage object detection, High accuracy with low inference time, Suitable for edge devices and field deployment, Handles multi-class ripeness stages effectively. And then CNN-based Lightweight Models used like MobileNet, Lightweight CNN, Mask R-CNN due to Low computational cost, Efficient on the mobile and IoT platforms. And then Transformer-based models (Emerging trend) used like RT-DETR, Vision Transformer, Attention-based YOLO due to better global context understanding, improved accuracy in complex backgrounds, still computationally heavy compared to YOLO. And then Traditional ML Models used like SVM, Random Forest, KNN due to it require manual feature extraction, Lower accuracy on image-based tasks.

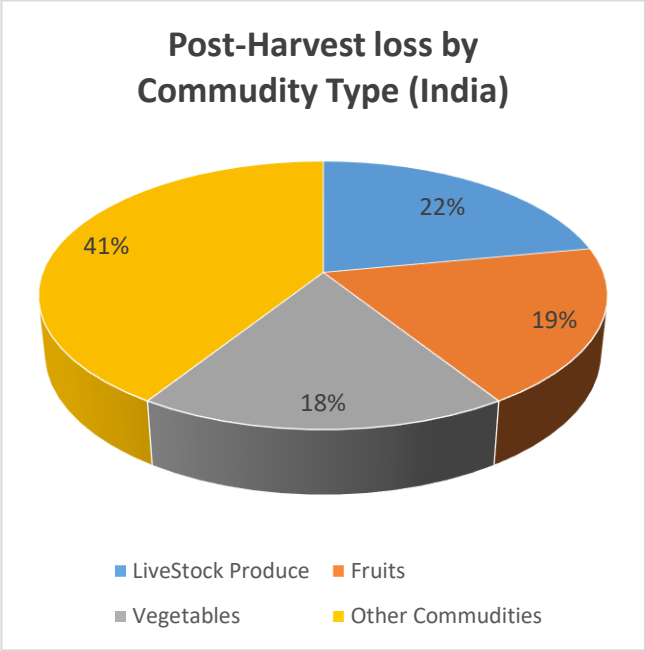

Figure 1.1: Post-Harvesting loss by Commodity type-India, 2022

The above image is showing the loss percentage in Post-Harvesting by the commodity type India in 2022 with the following percentages like Live Stock Products have 41%, Fruits have 19%, vegetables have 18%,

and other commodities have 22%. To contribute the losses of fruits nearly ₹29,000 crore annually in India. By using the SSD-MobileNetV2 (lightweight backbone + detector), YOLOv5/YOLOv6/YOLOv7(object detection models).

The characters that is used in the each model are fast inference and good accuracy, widely used in many real-world vision tasks, Learns features at different scales to localize objects of varying sizes,focuses on accuracy, speed, balancing the performance, excellent precision and feature learning for detecting fine difference between ripeness stages, slightly different model architecture which gets modified by comparing the YOLOv5 and YOLOv6, Designed for speed and efficiency, lower accuracy compared to YOLO model for SSD-MobileNetV2.

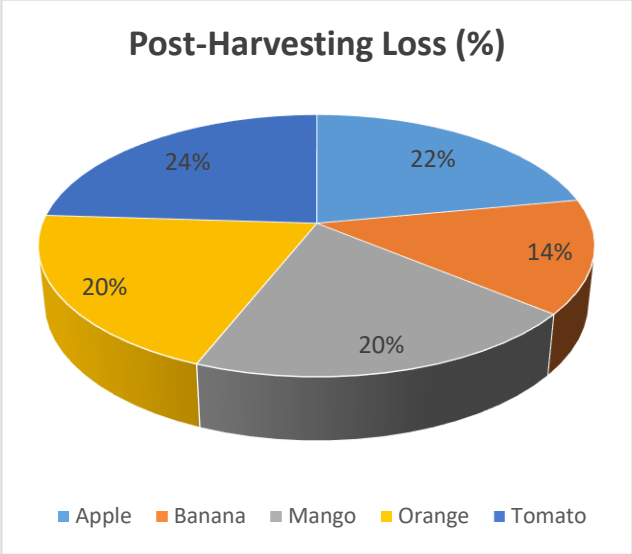

Figure 1.2: The above pie chart describe the loss of fruits that is having loss in Post-Harvesting.

The above image is showing the loss percentage in Post-Harvesting for the few fruits with the following percentages like Apple have 22%, Banana have 14%, Mango have 20%, Orange have 20%, tomato have 24%. The pie-chart percentages were derived by normalizing the average post-harvest loss values for only the fruits like-Apple, Banana, Mango, Orange, Tomato to represent their relative contribution to total losses within the group. Even though the actual loss for the above fruits are 11% for apple, 7% for banana, 10% for mango, 10% for orange, and 12% for tomato, these values are converted into proportional shares

when it is summed to 100% for visualization purposes.this type of normalization highlights the comparative impact of each fruit on overall post- harvest losses rather than absolute loss rates, with tomato and apple where these are contributing the highest shares due to their habitability and sensitive handling. It is taken between years of 2020-2024, which is primarily focusing on the India and similar tropical agricultural supply chain region. These averaged values are subsequently normalized to 100% to illustrate the relative contribution of each fruit to show the overall post-harvesting losses rather than depict losses for different specific years.

CHAPTER-2
LITERATURE SURVEY

2. Literature Survey

In 2023, research by Zhang and Xiao et al. highlighted the shift toward deep learning to overcome challenges like variable lighting and occlusions. Their work utilized CNNs with Attention Modules and multi-class ripeness classification models, proving that deep learning excels at local feature extraction while attention mechanisms improve global context understanding, particularly for fruits hidden in complex backgrounds. During this same period, Goh et al. (2024) explored the scalability of CNNs, emphasizing how end-to-end learning eliminates the need for manual preprocessing, thereby increasing the efficiency of the detection pipeline.

The year 2024 saw a focused effort on deployment and environmental integration. A. Authors et al. introduced a platform combining Lightweight CNNs with IoT sensor data (temperature and humidity) for spoilage detection in bananas. This approach demonstrated high working efficiency by fusing visual color/texture features with environmental data to predict shelf life. Simultaneously, Vo et al. (2024) utilized DenseNet architectures to handle dataset imbalances, ensuring the model remained robust even with sparse data.

By 2025, the field reached a peak in architectural optimization and multi-tasking capabilities. A. Authors et al. developed CES-YOLO specifically for blueberries; its optimized CNN backbone and Feature Pyramid Network (FPN) significantly enhanced feature extraction for small, densely clustered objects on edge devices. For strawberries, H. Yu et al. (2025) employed MobileNet and Depthwise Separable CNNs, which achieved an efficiency of 99.3% accuracy by reducing parameters and FLOPs, making them ideal for real-time mobile deployment. İ. Ünal et al. (2025) pushed the boundaries of multi-tasking by using a Lightweight Mask R-CNN with a ResNet-lite backbone, which successfully performed simultaneous ripeness detection and pest identification through pixel-level instance segmentation.

Furthermore, Wang et al. (2025) and other researchers integrated YOLOv9c and RT-DETR (Real-Time Detection TRansformer) backbones to improve gradient flow and feature reuse. These models, including EfficientNet and Inception, have demonstrated superior working efficiency by acting as powerful feature extractors that outperform traditional ML techniques, especially when utilizing Transfer Learning to maintain high performance even with small agricultural datasets.

2.1 Existing System

The existing system focuses on real-time multi-class fruit ripeness detection by integrating deep convolutional feature extraction with single-shot object detection frameworks. The system is designed to automatically detect fruits present in an image and simultaneously classify their ripeness stages into Unripe, Ripe, and Overripe categories.

In this approach, images are captured under real-world agricultural and laboratory conditions and provided as input to the system. The input images undergo a series of preprocessing operations, including resizing, normalization, and data augmentation, to improve robustness against variations in lighting conditions, background complexity, orientation, and scale. These preprocessing steps ensure stable and consistent feature learning during model training.

Feature extraction is carried out using lightweight convolutional neural network (CNN) backbones, particularly MobileNetV2, which efficiently learns discriminative features related to color, texture, and shape—key indicators of fruit ripeness. The extracted features are then passed to single-shot object detection models, namely YOLO (You Only Look Once) and SSD-MobileNetV2, which perform fruit localization and ripeness classification in a single forward pass.

YOLO is employed due to its high detection speed and real-time performance, making it suitable for field-level agricultural applications. SSD-MobileNetV2 is utilized for its computational efficiency and low memory requirements, enabling deployment on edge devices with limited processing power. The use of MobileNetV2 as a backbone significantly reduces model complexity while maintaining reliable detection accuracy.

The system workflow is organized into three main stages: Dataset Creation, Model Development, and Evaluation & Deployment. During dataset creation, fruit images are manually annotated with bounding boxes and ripeness labels based on expert knowledge, ensuring high-quality ground truth data. Model development involves preprocessing, model selection, hyperparameter tuning, and training of detection networks. Finally, model evaluation is performed using standard performance metrics such as accuracy, precision, recall, F1-score, and mean Average Precision (mAP) to assess detection reliability and classification effectiveness.

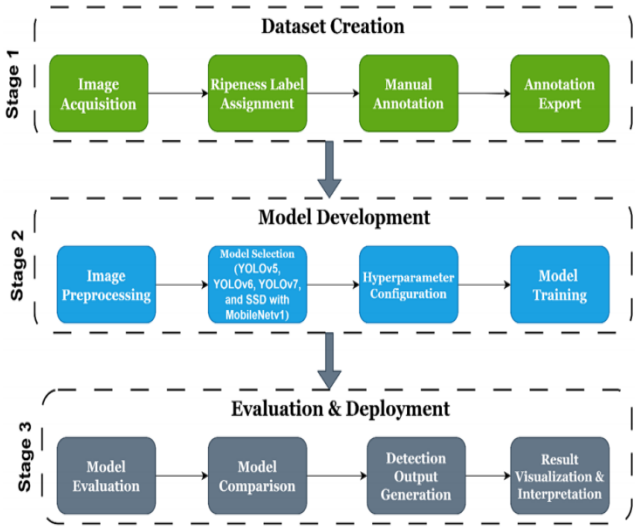

Figure 2.1: The image is the proposed frameworks that works in the existing method.

2.2 Proposed System

The proposed system introduces an EfficientNetB0-based intelligent fruit ripeness detection framework designed to accurately classify fruits into Unripe, Ripe, and Overripe stages under real-world agricultural conditions. Unlike traditional object-detection-driven pipelines, the proposed approach emphasizes deep visual reasoning and global ripeness understanding, making EfficientNetB0 the core visual backbone of the system.

The system begins with input image acquisition, where fruit images captured using mobile phones, digital cameras, or uploaded files are accepted without strict constraints on illumination, background, or image quality. This design choice ensures the applicability of the system in real farm environments. The input images are resized, normalized, and statistically aligned with the ImageNet distribution to maintain compatibility with the pre-trained EfficientNetB0 model.

At the core of the system lies the Ripeness Detection Model, which utilizes EfficientNetB0 as the primary feature extraction and reasoning backbone. The stem convolutional layer captures low-level visual cues such as color intensity, edges, and brightness, while successive Mobile Inverted Bottleneck (MBConv) blocks progressively extract higher-level semantic features related to fruit ripeness, including texture uniformity, surface dullness, bruising, and maturity degradation. Through depth-wise separable convolutions, squeeze-and-excitation mechanisms, and compound scaling, EfficientNetB0 achieves an optimal balance between accuracy and computational efficiency.

Transfer learning is employed by freezing the lower layers of EfficientNetB0 to retain generalized visual knowledge, while the higher layers are fine-tuned to specialize in ripeness-related features. As the network depth increases, spatial resolution decreases and semantic richness improves, allowing the model to focus on meaningful global ripeness indicators rather than localized pixel-level noise. Global Average Pooling aggregates feature activations across the image, reinforcing the concept that fruit ripeness is a global attribute rather than a localized phenomenon.

The extracted global features are passed through fully connected dense layers with dropout regularization to enhance robustness and prevent overfitting. A Softmax classification head generates normalized probabilities for the three ripeness classes, enforcing the biological constraint that a fruit can belong to only one ripeness state at any given time. The final ripeness prediction is selected based on the highest confidence score, which is also communicated to the user as an interpretable confidence percentage.

Beyond classification, the proposed system incorporates a Decision Support Layer that translates ripeness predictions into actionable storage and handling recommendations. Unripe fruits are suggested for room-temperature storage to facilitate ripening, ripe fruits for immediate consumption or short-term refrigeration, and overripe fruits for cold storage or processing. This transforms the AI model from a mere classifier into a practical agricultural assistant.

A Web-based Human–System Interaction Layer enables users to upload fruit images and visualize predictions, confidence levels, and storage advice in a clear and user-friendly manner. The complexity of the underlying AI processes is abstracted away, improving accessibility and user trust. A feedback loop connects the prediction engine with the interface, ensuring seamless interpretation of model outputs into meaningful user actions.

Model training is optimized using categorical cross-entropy loss, class imbalance compensation, Adam optimization, learning rate scheduling, batch normalization, Swish activation functions, and L2 regularization. These mechanisms collectively improve convergence stability, generalization capability, and robustness against dataset imbalance and visual variability.

Performance evaluation is carried out using standard metrics such as accuracy, precision, recall, AUC, and cross-entropy loss, ensuring reliable assessment of classification quality, confidence separability, and decision consistency. The results demonstrate that the proposed EfficientNetB0-based system provides an accurate, scalable, and computationally efficient solution for fruit ripeness detection, making it suitable for real-time agricultural, post-harvest, and storage decision-making applications.

2.3 System Study

The system study focuses on understanding the data source, dataset characteristics, tools, technologies, and processing workflow employed for fruit ripeness detection. The proposed system leverages publicly available datasets and widely adopted machine learning frameworks to ensure reproducibility, scalability, and practical relevance.

Kaggle serves as the primary platform for dataset acquisition and experimentation. It is a globally recognized repository that brings together beginners and professional data scientists, offering access to real-world datasets, machine learning competitions, and collaborative learning opportunities. Kaggle enables users to download datasets, experiment with models, share notebooks, and gain insights from community feedback. This platform is particularly valuable for building practical machine learning solutions and developing portfolios through hands-on experience.

For this system, the "Fruit Ripeness: Unripe, Ripe, and Rotten" dataset from Kaggle is utilized. This dataset is a labeled image collection designed for supervised learning tasks in computer vision. It contains thousands of images of fruits such as apples, bananas, mangoes, oranges, and tomatoes, each categorized into three ripeness stages: Unripe, Ripe, and Overripe/Rotten. The dataset structure supports direct use in training classification models and is well-suited for agricultural automation and post-harvest quality assessment.

The dataset captures visible variations in color, texture, size, and shape across different ripeness stages. For example, unripe fruits typically exhibit green coloration and firm texture, ripe fruits show vibrant colors with smooth or slightly soft surfaces, and rotten fruits display dark patches, soft or mushy textures, and irregular shapes. These visual attributes form the key features that the machine learning model learns during training. The diversity of samples enhances the robustness of the trained model and improves its generalization to real-world scenarios.

The system employs TensorFlow and Keras as the primary deep learning frameworks for model development and training. Transfer learning is adopted using pre-trained convolutional neural networks such as VGG16, ResNet, MobileNet, or EfficientNet, which are originally trained on large-scale datasets like ImageNet. By leveraging these models as feature extractors, the system significantly reduces training time and improves performance, even with limited domain-specific data.

Image preprocessing and enhancement are handled using OpenCV, which plays a crucial role in preparing the dataset for model training. Preprocessing operations include image resizing to a uniform resolution,

pixel value normalization, color space handling (RGB or grayscale), and noise reduction. In addition, data augmentation techniques such as rotation, flipping, zooming, shifting, and color variation are applied to artificially increase dataset diversity. This improves the model's ability to handle real-world variations in lighting, orientation, and background.

Data handling and processing are performed using NumPy and Pandas, which facilitate efficient numerical computation and structured data management, respectively. Dataset splitting into training and testing subsets is carried out using Scikit-learn, ensuring reliable evaluation of model generalization. Scikit-learn also provides access to standard evaluation metrics such as accuracy, precision, recall, and F1-score, which are essential for measuring classification effectiveness.

Model training involves fine-tuning pre-trained networks on the fruit ripeness dataset. Training strategies such as model checkpointing, early stopping, and learning rate scheduling are employed to prevent overfitting and ensure stable convergence. These techniques allow the system to retain the most optimal model version and adapt learning behavior during training for improved accuracy.

Finally, system evaluation is conducted using confusion matrices and classification reports, which provide detailed insights into class-wise prediction performance. These tools help identify misclassification patterns between unripe, ripe, and overripe categories. While overall accuracy offers a general performance measure, per-class metrics such as precision and recall are particularly important in ripeness detection applications, where accurate identification of ripe fruits is critical for harvesting and storage decisions.

Overall, the system study highlights a well-structured integration of public datasets, deep learning frameworks, image processing tools, and evaluation techniques, forming a reliable foundation for effective fruit ripeness detection using computer vision.

CHAPTER-3
METHODOLOGY

3. Methodology

Based on the existing methodology in this section giving a detailed explanation for the methodology that being used in this paper that is about EfficientNetB0. It is going to act as the visual reasoning backbone of the proposed system for ripeness detection which is responsible to transform the raw fruit images into meaningful ripeness related feature.

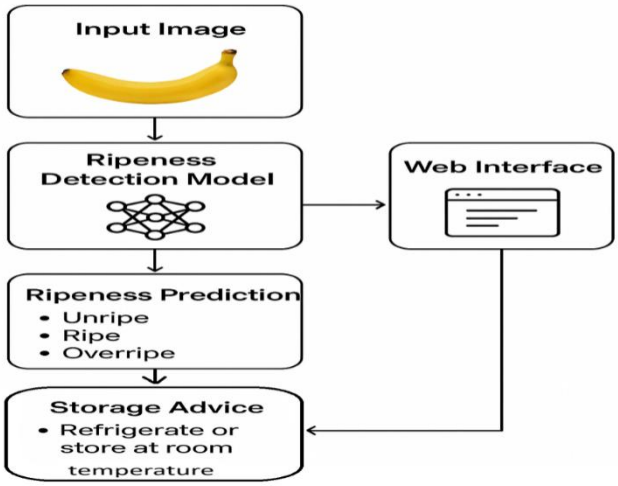

Figure 3.1: The image is block diagram of the system working

❖ Input image

The input image acquisition module serves as the entry point for the ripeness detection process, whereby real images of fruits, acquired from mobile phones, digital cameras, or from files, can be provided by the users. In this case, since there are no restrictions regarding illumination, background, and quality of the images, this process can effectively suit real farm conditions.

❖ Ripeness Detection Model(Core Intelligence Unit)

The Detection Model is essentially the brain of the entire system as it works on analyzing the fruit images uploaded onto the system for ripeness. The model uses EfficientNetB0 for the purpose of feature extraction and is used for standardizing the images and analyzing features such as color and texture.

❖ Ripeness Prediction Output

In this respect, the model delivers the ultimate ripeness classification, which can either be Unripe, Ripe, or Overripe, based on the confidence levels of each class instead of fixed thresholds. Unripe means the fruit is immature, Ripe means the fruit is perfectly ripe for harvest or eating, and

Overripe means the fruit has started to degenerate in its ripeness due to the natural fruit-ripening process.

❖ Storage Adivce Generation (Decision Suport Layer)

This level of translation provides an interpretation of the maturity degree into post-harvest handling and storage. The fruits that are unripe can be stored at room temperature for ripening, ripe fruits can be stored for short-term refrigerated storage or for immediate consumption, while overripe fruits can be stored in the refrigerator or processed.

❖ Web Interface (Human-System Interaction Layer)

It acts as the interface between a human and the system by allowing users to upload images and, in turn, view predictions and storage advice. It represents clear presentation of the ripeness labels and levels of confidence and recommendations, hiding AI complexities for better access and building user trust in the system outputs.

❖ Feedback Loop Between Model and Interface

The feedback mechanism brings the model together with the web interface and allows the AI system to work as a holistic decision-making assistant. The predictions are translated into concise storage recommendations, thereby establishing a connection between artificial intelligence output and user understanding.

From the block diagram provided, it can be noted that EfficientNetB0 takes the resized and normalized input image to extract the features related to ripeness. While the stem convolutional layer in EfficientNetB0 captures low-level characteristics such as color and edges, the subsequent Mobile Inverted Bottleneck layers are responsible for the extraction of higher-level characteristics like texture, bruise, and uniformity.

In this architecture, the higher levels of the pre-trained EfficientNetB0 model are fine-tuned to specialize in ripeness features, holding onto the general image knowledge learned for the ImageNet dataset. Redundancy removal by MBConv layers, global average pooling to encode ripeness features compactly, and density layers to generate classifications along with confidence values result in the accurate, efficient, and scalable nature of the EfficientNetB0 model. The clear explanation of the EfficientNetB0 working is shown with below architecture:

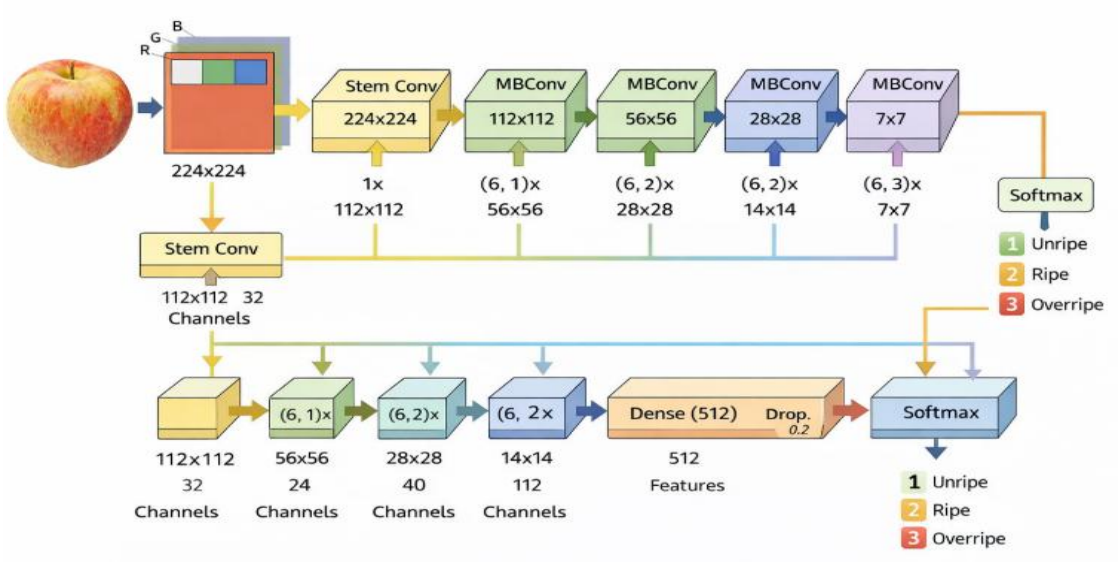

Figure 3.2: The architecture of the EfficientNetB0 which is used in the model training.

The steps that are used in the model training by using EfficientNetB0 is described in the below with certain steps as follows:

❖ Input Fruit Image (RGB → 224×224)

The normalization of the fruit image is then accomplished through this step in order to convert the image into a standardized size of 224x224 and then split the image into its red, blue, and green components, as the red is associated with the maturity of the fruit, the green indicates the decay associated with chlorophyll, and the blue indicates the reflectance and bruising.

❖ Stem Convolution (Initial Visual Encoding)

It is the first stage where the input image is transformed into a feature map that measures 112×112 pixels and comprises 32 channels, and the process extracts raw features of the visual world, including edges, colors, and brightness variations. It also normalizes lighting and image quality, allowing accurate features to be extracted that are used by the EfficientNet model for ripeness evaluation.

❖ MBConv Block - Stage 1 (112×112 → 56×56)

This leads to a feature map size of 56x56 pixels with an increase in channel depth to 24, thereby improving feature extraction capabilities. The model is able to extract pre-ripeness signs such as color variations and surface patterns of fruits and vegetables, where a six-fold expansion is applied to examine high-dimensional features before compressing important pre-ripeness features for effective detection.

❖ MBConv Block - Stage 2 (56×56 → 28×28)

The feature map now is 28×28 with 40 channels, allowing the model to grasp more abstract features of the ripeness, such as saturation, texture, and patch-level cues. While the spatial details are reduced at this stage, greater semantic resolution of the network on ripeness would enhance the sensitivity toward meaningful patterns rather than the exact positions of each pixel.

❖ MBConv Block - Stage 3 (28×28 → 14×14)

By this stage, the feature map is down to 14×14 with 112 channels; hence, the model will be able to capture complex signs of ripeness such as texture deterioration, dullness, and loss of shine. This is the trade-off between spatial detail and semantic representation: Although the former decreases, the latter strengthens, thereby enabling the accurate detection of overripe characteristics.

❖ MBConv Block - Final Stage (14×14 → 7×7)

"At this point, the feature map is reduced to 14x14 spatial dimensions but now has 112 channels. This allows us to capture complex ripeness cues like texture degradation, maturity, and loss of shine." Spatial resolution is reduced, but semantic encoding improves, leading to reliable identification of over-ripeness. During this level, the size of the feature map is reduced from 14×14 to 7×7, and this is where the shift from image understanding to reasoning occurs. Indeed, details at this point are reduced to allow the model to extract effective abstract ripeness indicators based on earlier color and textural features.

❖ Global Features Flow (Bottom Path)

This represents the lower pipeline in EfficientNet, whereby the depth of the channels grows from 32 to 24, 40, and then 112 as the network becomes more complex. Despite the reduction in the spatial dimensions, the semantic interpretation of the ripeness of fruits improves as the feature vectors are compressed.

❖ Dense Layer (512 Features)

This layer combines 512 features into a single decision-ready representation of the maturity of fruits. Applying dropout at a rate of 0.2 enhances robustness since the model is prevented from being reliant on any given color and texture indicator, whatever it may be.

❖ Softmax Classification Head

The classification head is used to generate Normalized Probabilities of the three mutually exclusive classes of ripeness – unripe, ripe, and overripe. The model's confidence on each state and

the biological constraint of a fruit only being able to be at one level of ripeness at any given time is captured through the softmax competition.

❖ Confidence Output (UI Meaning)

The confidence output shows the system's confidence in the predicted ripeness class, computed as the maximum value from the softmax probabilities. Presenting this as an interpretable confidence level allows the user to judge the reliability of the prediction and decide whether to act on it automatically or perform a manual re-inspection.

Mathematical Foundation

❖ Image Conditioning Operator

This image conditioning operator takes the uploaded RGB image and readies it for the features by resizing it into a format that can be handled by the EfficientNet model. This ensures that the model receives standard input for accurate results.

$$T(I) = \Psi(\Omega(I))$$

Where as:

- $\Omega(.) \rightarrow$ enforces spatial conformity (256×256)
- $\Psi(.) \rightarrow$ enforces statistical compatibility with ImageNet

Step 2 involves statistical alignment of the model's training distribution on the ImageNet dataset by normalizing the color intensity to the standards of the pretrained model. This handles the issue of misleading information derived from visual variability.

$$\Psi(I_C) \rightarrow \textit{distribution-aligned intensity}$$

❖ Transfer Feature Projection (EfficientNetB0 Backbone)

The feature projection step involving the transfer learning capability employs the EfficientNetB0 as the feature extractor to project the conditioned image into a semantics space that is ripeness-aware, instead of carrying out the classification task. The low-level feature layers are frozen in order to retain the generalized features of the pre-trained model, such as the edges and textures.

$$F_{ripeness} = \Phi_{B0}(I_{cond})$$

Where as:

- Φ_{B0} is a frozen-to-adaptive projection
- Only the uppermost representational layers are adjustable

Adaptation is applied to the high semantic layers, allowing the model to specialize in features related to ripeness. In this way, general vision knowledge that is retained balances with focused learning of the representation for accurate perception of fruit ripeness.

$$UpdateAllowed(l) = \begin{cases} False, & l \in generic\ visual\ layers \\ True, & l \in ripeness\text{-}specialized\ layers \end{cases}$$

❖ Spatial Evidence Consolidation (Global Averaging)

This stage integrates evidence for ripeness by averaging their activations over each feature channel, independent of the precise locations of color or texture patterns. This produces a vector for ripeness evidence over the whole image, treating all indicators found over the regions of fruits equally.

$$e_c = mean\ activation\ strength\ of\ channel\ c$$

Such an approach is also supported by the fact that the ripeness of the fruit is a worldwide phenomenon, as opposed to being local. This means that every change or defect, no matter where it occurs, impacts the ripeness of the fruit.

$$E = [e_1, e_2, ..., e_C]$$

❖ Distribution Re-Centering Layer (Batch Normalization)

Batch normalization re-centers the global ripeness evidence for stabilization of variations w.r.t. different cameras, lighting, and backgrounds. Normalization decreases the noise caused by the radiance; therefore, the model produces reliable predictions on the classes of ripeness.

$$E \rightarrow standardized\ responsestate$$

After standardization, the scale and shift parameters are used to add flexibility to the classification process by enabling adaptability yet stability in different conditions for the inputs to the network.

$$Output = scale\text{-}adjusted(E) + bias\text{-}shift$$

❖ Information Randomization Mechanism (Dropout Cascade)

Dropout adds a controlled form of randomization of information to feature vectors during training by partially hiding them, thus imitating incomplete visual evidence. This helps the networks learn to make predictions using multiple indicators of ripeness.

$$E' = feature\text{-}wise\ \ survival\ process$$

By doing so, it avoids leaning too heavily on one visual feature like color, texture, or brightness, thus allowing a model to make a decision through a majority vote of several visual cues.

The dropout process is essential in that it promotes stable ripeness decisions, irrespective of poor visual conditions.

❖ Smooth Ripeness Transition Function (Swish Activation)

The smooth transition function for the state of ripeness uses an Swish activation to support smooth, continuative, gradual feature responses aligned with the natural process of ripening. Unlike ReLU, Swish allows partial activation of negative values, preserving subtle evidence about ripeness and allowing smoother transitions from unripe to ripe and overripe states.

$$activation\ strength = input \times self\text{-}gated\ openness$$

The activation function reflects the smoothness and continuity of fruit ripening. By retaining partial evidence instead of removing it, this function preserves the subtle ripeness cues that are needed for reliable and accurate classification.

❖ Confidence Constraint Mechanism (L2 Regularization)

The confidence constraint applies L2 regularization to suppress overly high confidence values for particular fruit looks instead of general ripeness distributions. The quadratic terms add complexity and penalize large weights.

$$Complexity\ Cost\ \alpha\ \ parameter\ magnitude$$

This acts as a regularizer encouraging the optimizer to seek an explanation of the patterns in terms of ripeness that is as simple as possible and resists the temptation toward overconfidence. Given the small and diverse datasets of fruits, these constraints help prevent overfitting and allow the model to generalize better.

❖ Competitive Ripeness Scoring (Softmax Head)

This ‘competitive ripeness scoring’ layer relies on a softmax head where the classes, unripe, ripe, and overripe, are made to compete among each other. This process of preference scores being converted into normalized belief strengths means that the more the confidence level increases, the less the confidence level of the other stages.

$$S = [OverripeScore, RipeScore, UnripeScore]$$

The final prediction chooses the class with the highest confidence, satisfying the biological constraint that the fruit can be at only one ripeness level at a time. This ensures that the model gives a distinct output regarding the ripeness of the fruit.

❖ Dataset Imbalance Compensation Rule

The imbalance compensation rule is a method for handling class imbalances in fruit image datasets, as underripe fruits are typically less represented. The underrepresented ripeness stages get higher penalties for loss in the model, as the model will try to put more focus on such stages.

Loss Impact α rarity of class

It might have been the case that without imbalance compensation, achieving a high level of accuracy would have resulted in training a network to focus on the majority class, leading to inefficiency in detecting overripe fruits in practical applications.

❖ Learning Rate Evolution Strategy

The learning rate evolution strategy follows a two-fold process, reflecting the process by which confidence matures over time. A cautious phase of training follows, followed by a reduction in the learning rate during the refinement phase, ensuring a precise and stable update of the model..

Learning aggressiveness = time-aware control signal

This is a time-aware control signal because the slow adaptation in the beginning prevents the features from being instable, and the gradual adaptation in the end allows a fine distinction between the levels of ripeness.

❖ Adaptive Optimization Dynamics (Adam)

This system relies on the Adam Optimizer, which adjusts each parameter independently according to its learning stability. This means that those learning more steadily get more intense learning, while those learning more erratically get reduced learning if these are the features being learned. Such features include color and texture.

$$S_k(t) = \textit{temporal consistency of } g_k$$

$$\mathrm{M}_k(t) = \textit{magnitude tendency of } g_k$$

This enables the reliable information such as ripeness indicated by color maturation to converge rapidly, while the noisy information like the texture irregularities learns at a slower pace. Together with the learning rate schedule, it promotes the reliable features by adjusting the update process in a self-correcting fashion.

$$\triangle\theta_k(t) = \eta(t).\frac{S_k(t)}{\sqrt{\mathrm{M}_k(t)} + \epsilon}$$

$$\theta_k(t+1) = \theta_k(t) - \triangle\theta_k(t)$$

Where as:

- θ_k = k-th trainable parameter (color filter, texture detector, etc.)
- $g_k(t)$ = gradient signal observed at epoch t.

❖ Evaluation Metrics -- System-Centric Mathematical Formulation

a. Accuracy- Belief-Reality Agreement Ration

Accuracy reflects the degree to which the best predictions conform to the actual ripeness class, which indicates the general level of decision-making capability, regardless of the sensitivity to the classes themselves. Accuracy can be considered a sanity check for the convergence of learning but can be misleading, particularly where there are imbalances, in that it indicates success without stating its causes.

$$A_{ripeness} = \frac{1}{N}\sum_{i=1}^{N} I\left(arg\ max(P_i) \equiv y_i\right)$$

Where as:

- P_i = predicted confidence distribution
- y_i = true ripeness state
- $I(.)$ = belief match indicator

b. Precision-Claim Reliability Index

Precision then refers to the reliability of a system in predicting whether instances belong to a specific class of ripeness, such as overripe fruits. High precision would minimize false alarms, something critical for both harvest and storage decisions, while drops in precision hint at possible miscalibration or overconfidence in the model.

$$Q_c = \frac{\sum_i I\left(\hat{y}_i = c \wedge y_i = c\right)}{\sum_i I\left(\hat{y}_i = c\right)}$$

c. Recall-Ripeness Coverage Capacity

Recall tests how well a system can detect all cases of a given ripeness condition rather than focusing on prediction accuracy. High recall values are particularly important when detecting rare but very important states, such as an overripe fruit, to guarantee successful handling of class weighting and imbalance.

$$\mathrm{R}_c = \frac{\sum_i I\left(\hat{y}_i = c \wedge y_i = c\right)}{\sum_i I\left(y_i = c\right)}$$

d. AUC-Confidence Separability Measure

AUC is "the capacity to correctly rank examples according to confidence, without concern for any particular decision threshold." It "captures the quality of confidence as being higher for positives than for negatives, and can be used to compare training processes or architectures in a reliable fashion."

$$U_c = Pr\left(P_c(x^+) > P_c(x^-)\right)$$

$$U_c = \frac{1}{|X_c^+||X_c^-|}\sum_{x^+}\sum_{x^-} I\left(P_c(x^+) > P_c(x^-)\right)$$

❖ Categorical Cross-Entropy Loss

The model is using categorical cross-entropy loss for training. Where y_i is thr ground truth label, and $P(y_i)$ is predicted probability for class i. Minimizing loss function for ensuring accurate classification across all ripeness stages.

$$L = -\sum_{i=1}^{C} y_i \log\left(P(y_i)\right)$$

❖ Softmax Classification Function

The Softmax function is for converting raw logits into class probabilities. Where z_i is representing the logit value for class i, and C is representing the total number of ripeness classes.

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

❖ Confidence Communication Logic (UI)

The UI reflects a confidence value as a single entity—which is derived from a maximum softmax score normalized to 100—to the exclusion of a probability distribution.Agricultural consumers can readily judge how much confidence there is in a given maturity statement through this interface system that associates a level of confidence with a corresponding action—for instance, when confidence is high, an action is taken; when confidence is low, a re-confirmation is necessary.

$$C_{display} = \max_c(P_c) \times 100$$

CHAPTER-4
SYSTEM DESIGN

4. System Design

4.1 System Requirements

The proposed fruit ripeness detection system is designed to operate using modern machine learning frameworks and publicly available datasets. To ensure accurate training, evaluation, and deployment of the model, the following system requirements are identified based on the dataset characteristics, model development pipeline, and supporting technologies.

1. Dataset Requirements

The system requires a **labeled image dataset** containing fruit images categorized into **Unripe, Ripe, and Rotten/Overripe** classes. The dataset used is obtained from **Kaggle**, which provides thousands of real-world fruit images including apples, bananas, mangoes, oranges, and tomatoes. Each image is labeled for supervised learning and reflects variations in **color, texture, size, and shape**, enabling robust ripeness classification. The dataset must support directory-based class separation suitable for deep learning frameworks.

2. Software Requirements

The implementation of the system requires the following software tools and libraries:

- **Programming Language:** Python
- **Deep Learning Frameworks:** TensorFlow and Keras for model building, transfer learning, training, and inference
- **Image Processing Library:** OpenCV for image resizing, normalization, color space handling, and enhancement
- **Numerical Computing:** NumPy for efficient numerical operations
- **Data Handling:** Pandas for managing image paths, labels, and metadata
- **Machine Learning Utilities:** Scikit-learn for dataset splitting (training/testing) and evaluation metrics
- **Visualization & Evaluation:** Matplotlib and Seaborn (optional) for plotting confusion matrices and performance graphs

These tools collectively support the full pipeline from data preprocessing to model evaluation.

3. Model and Training Requirements

The system relies on **transfer learning** using pre-trained convolutional neural networks such as **VGG16, ResNet, MobileNet, or EfficientNetB0**, which are trained on large-scale datasets like ImageNet. The pre-

trained models serve as feature extractors, reducing training time and improving accuracy with limited data.

Model training requires:

- Image resizing to a uniform input size
- Pixel value normalization
- Data augmentation techniques such as rotation, flipping, zooming, shifting, and color variation
- Fine-tuning of higher layers while freezing lower layers
- Use of callbacks such as **Model Checkpointing**, **Early Stopping**, and **Learning Rate Scheduling** to prevent overfitting and enhance convergence

4. Hardware Requirements

To support efficient model training and experimentation, the following hardware configuration is recommended:

- **Processor:** Multi-core CPU (Intel i5/i7 or equivalent)
- **Memory (RAM):** Minimum 8 GB (16 GB recommended)
- **Storage:** At least 10–20 GB free disk space for datasets and model checkpoints
- **GPU (Optional but Recommended):** NVIDIA GPU with CUDA support for faster training

For small-scale experiments, the system can also be executed on cloud platforms such as **Kaggle Kernels** or **Google Colab**.

5. Evaluation and Performance Requirements

The system must support detailed performance evaluation using:

- **Accuracy** for overall correctness
- **Precision, Recall, and F1-score** for class-wise reliability
- **Confusion Matrix** to analyze misclassifications across ripeness stages

These metrics ensure the model performs reliably, especially in distinguishing critical ripeness stages such as ripe and rotten fruits.

6. Usability and Deployment Requirements

The trained model should be deployable in a user-friendly environment where users can upload fruit images and receive ripeness predictions. The system must support clear interpretation of results, including confidence levels, enabling its use in agricultural automation, produce sorting, and quality assessment applications.

4.2 UML Diagrams:

UML diagrams are used to represent the structural and behavioral aspects of the AI-Powered System for Ripeness Detection and Post-Harvest Storage Advice. These diagrams help in understanding the interaction between the user and the system, the internal structure of the system, and the sequence of operations.

The following UML diagrams are used in this system:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- Deployment Diagram

4.2.1 Use Case Diagram:

The use case diagram illustrates the interaction between the **user** and the **system**.

Actor

- User (Farmer / Operator)

Use Cases

- Capture / Upload Fruit Image
- Preprocess Image
- Detect Ripeness
- Generate Storage Advice
- View Results

The user uploads or captures a fruit image. The system processes the image, detects the ripeness level, and provides suitable post-harvest storage advice. The final result is displayed to the user.

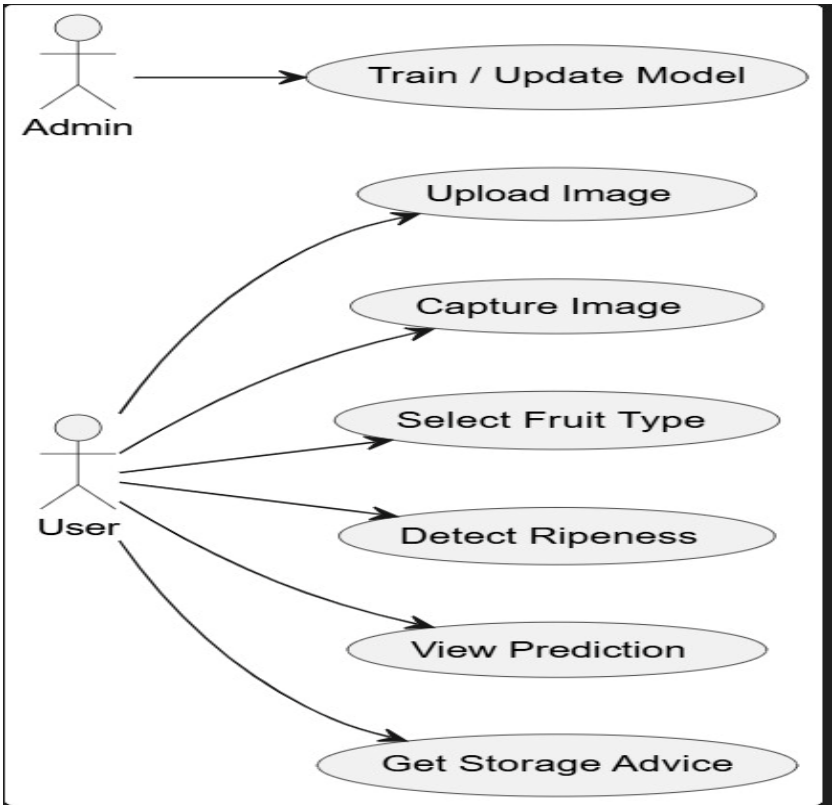

Fig:4.1 Use Case Diagram

4.2.2 Class Diagram:

The class diagram represents the internal structure of the system and the relationship between different classes.

Classes Used

- ImageInput
- ImagePreprocessing
- FeatureExtraction
- RipenessClassifier
- StorageAdvisor
- ResultDisplay

Each class contains attributes and methods related to its functionality. The classes are connected in such a way that the output of one class becomes the input of the

next class.

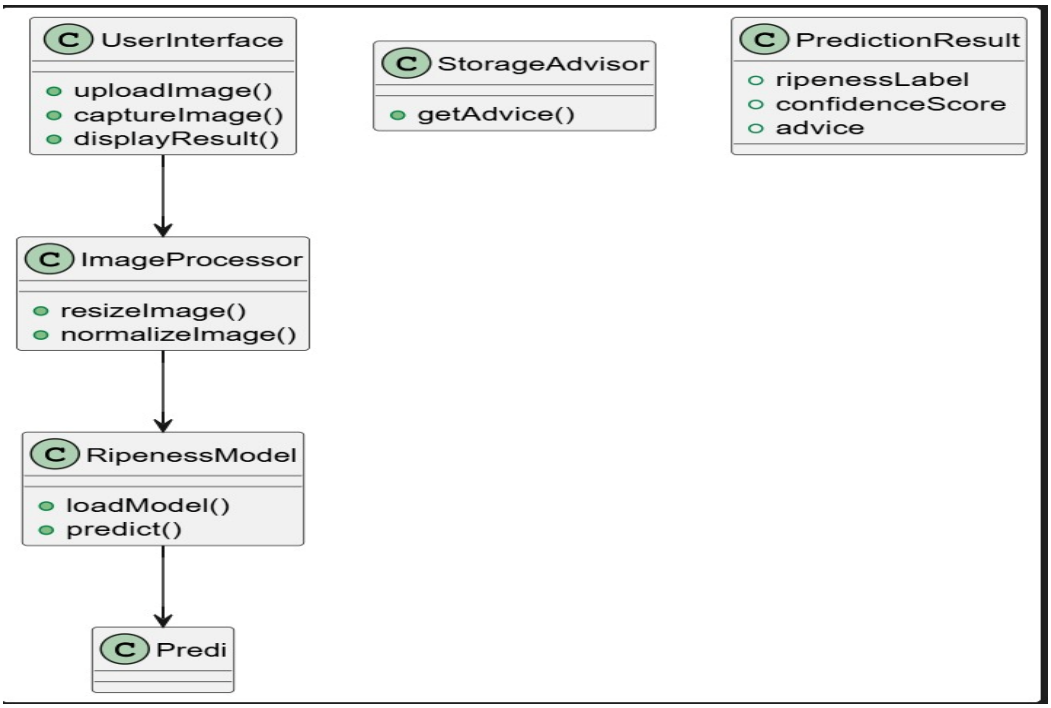

Fig:4.2 Class Diagram

4.2.3 Sequence Diagram:

The sequence diagram shows the step-by-step interaction between system components over time.

Sequence of Operations

1. User uploads fruit image
2. Image is sent to preprocessing module
3. Features are extracted from the image
4. Ripeness is detected using classifier
5. Storage advice is generated
6. Result is displayed to the user

This diagram explains how the system processes the image sequentially from input to output

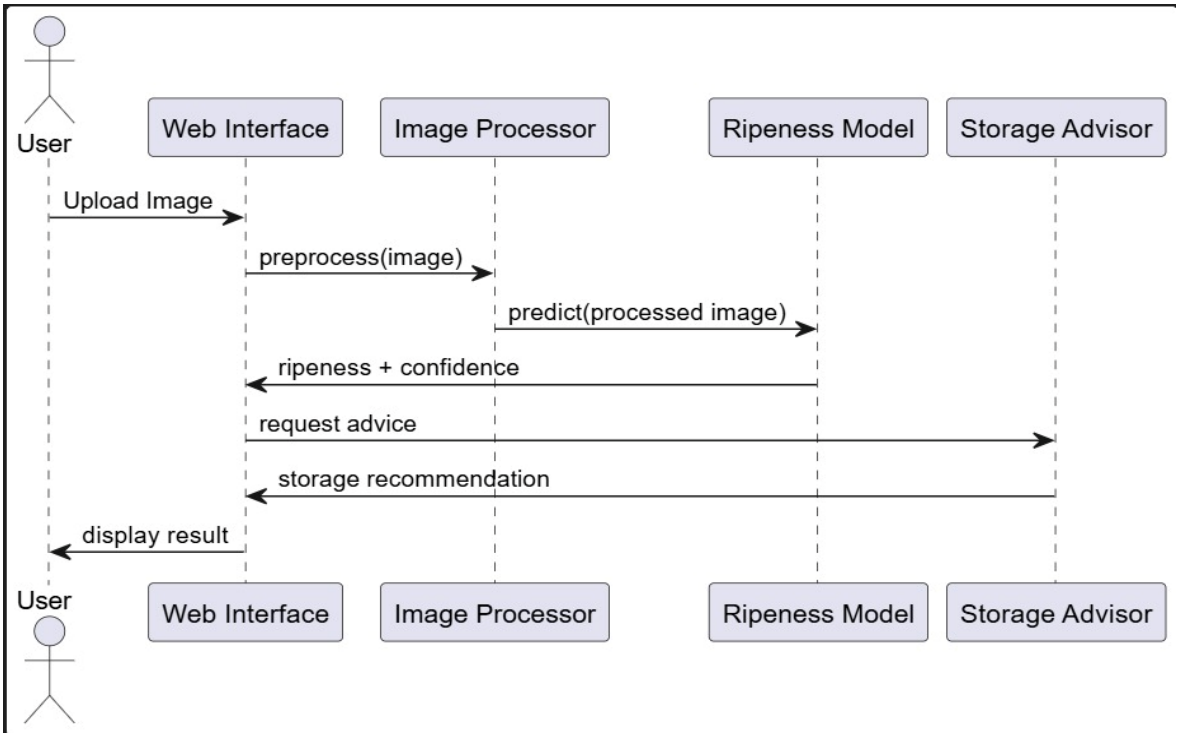

Fig:4.3 Sequence Diagram

4.2.4 Activity Diagram

The activity diagram shows the step-by-step flow of operations in the fruit ripeness detection system.

Sequence of Operations

1. The user uploads a fruit image to the system.
2. The uploaded image is preprocessed to improve quality and prepare it for analysis.
3. The system detects the ripeness level of the fruit from the processed image.
4. If the fruit is identified as unripe, room temperature storage is recommended.
5. If the fruit is identified as ripe, refrigeration is suggested; if it is overripe, immediate use is advised.
6. The process ends after providing the appropriate storage or usage recommendation to the user.

This diagram explains how the system processes the fruit image step by step and guides the user with suitable actions based on ripeness.

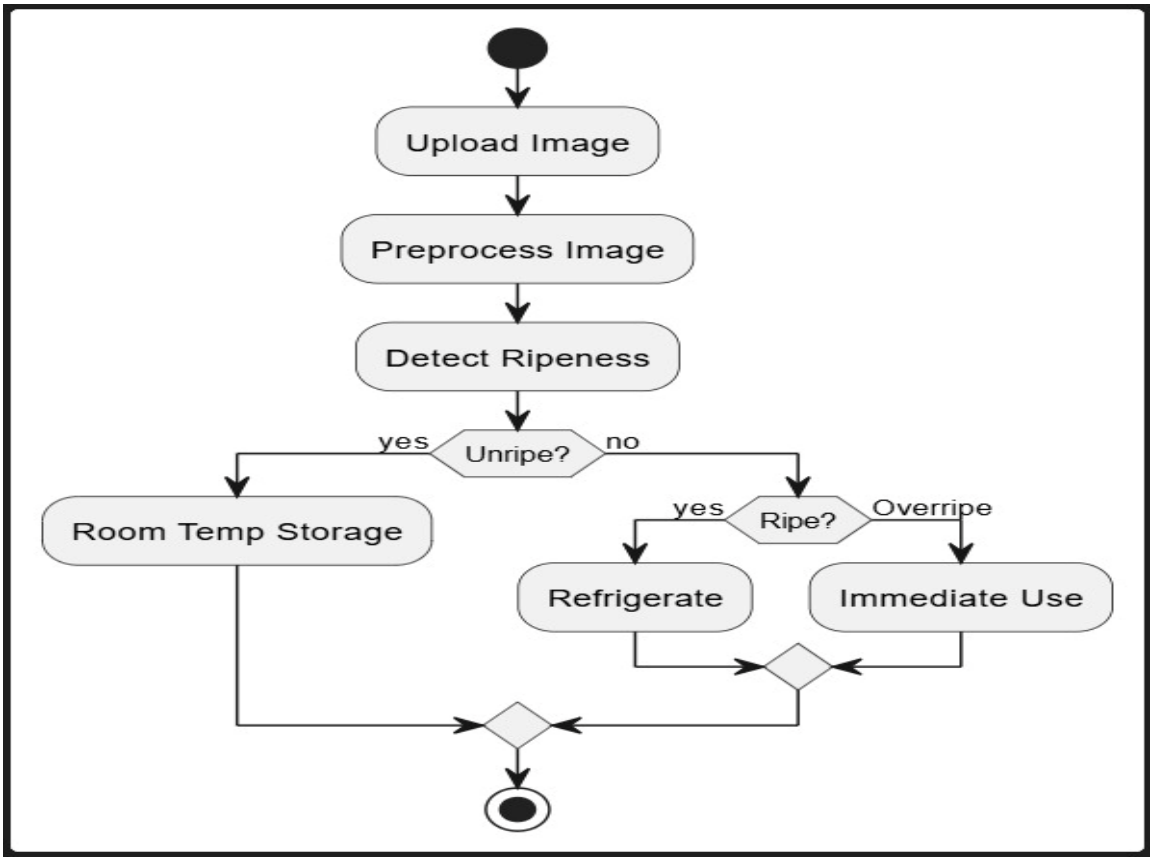

Fig:4.4 Activity Diagram

4.2.5 Deployment Diagram:

The deployment diagram shows the physical architecture of the system and how software components are deployed across hardware and environments.

Sequence of Deployment

1. The user accesses the application through a web browser or mobile interface.
2. The frontend application is deployed on a client device and handles image upload and result display.
3. The uploaded image is sent to the backend server for processing.
4. The backend hosts the image preprocessing module and ripeness detection model.
5. The trained machine learning classifier is deployed on the server to analyze the image and predict ripeness.
6. The processed results and storage recommendations are returned to the frontend and displayed to the user.

This diagram explains how different system components are deployed and interact across client and server environments to deliver fruit ripeness detection results.

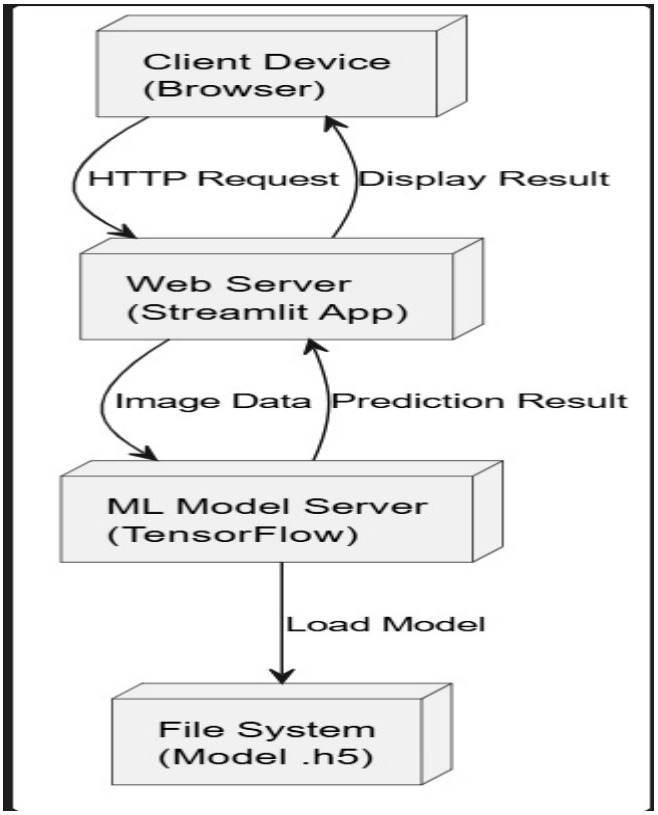

Fig:4.5 Deployment Diagram

4.3 Implementation:

The implementation of the proposed fruit ripeness detection system is carried out by integrating deep learning–based visual reasoning with a user-oriented decision support interface. The core objective of the implementation is to translate raw fruit images into meaningful ripeness predictions using **EfficientNetB0 as the backbone model**, while ensuring robustness, interpretability, and practical usability in real agricultural environments.

The system implementation begins with the **input image acquisition module**, where users upload fruit images captured using mobile phones, digital cameras, or stored image files. No strict constraints are imposed on lighting conditions, background complexity, or image quality, enabling the system to function effectively under real-world farm and post-harvest conditions. Each uploaded image is processed independently, making the system scalable and suitable for real-time use.

Once the image is received, it undergoes **image conditioning and preprocessing**, which includes resizing to a standardized resolution compatible with EfficientNetB0 and normalization aligned with ImageNet statistics. This conditioning step ensures spatial and statistical consistency, reducing variability caused by differences in camera sensors, illumination, and environmental factors. The processed image is then forwarded to the core intelligence unit of the system.

The **ripeness detection model**, built upon the EfficientNetB0 architecture, forms the computational backbone of the implementation. EfficientNetB0 is employed as a transfer learning–based feature extractor, where the lower convolutional layers are frozen to retain general visual knowledge such as edge, color, and texture detection. The higher semantic layers are fine-tuned to specialize in fruit ripeness characteristics. Through its stem convolution and successive Mobile Inverted Bottleneck (MBConv) blocks, the model progressively transforms raw pixel data into high-level ripeness-aware representations. Early layers capture low-level visual cues such as color gradients and edges, while deeper layers encode complex features such as texture degradation, surface uniformity, bruising, and maturity indicators.

Following feature extraction, **global average pooling** is applied to consolidate spatial evidence across the entire fruit surface. This design choice aligns with the biological nature of fruit ripeness, which is a global property rather than a localized phenomenon. Batch normalization is then used to stabilize feature distributions across varying imaging conditions, improving generalization. Dropout regularization further enhances robustness by preventing over-dependence on individual visual cues and encouraging the model to learn from multiple indicators such as color, texture, and brightness.

The processed features are passed through a **dense layer with 512 neurons**, producing a compact, decision-ready representation of fruit maturity. A **softmax classification head** converts these features into normalized probability scores corresponding to the three ripeness classes: Unripe, Ripe, and Overripe. The competitive nature of the softmax function ensures that the model outputs a single dominant ripeness state, respecting the biological constraint that a fruit can exist in only one ripeness stage at a time.

To address real-world dataset imbalances—where certain ripeness classes such as unripe or overripe fruits may be underrepresented—the implementation incorporates **class imbalance compensation** through weighted loss functions. This ensures that minority classes receive adequate attention during training. Model optimization is performed using the **Adam optimizer**, which adapts learning rates for individual parameters based on gradient stability. A staged learning rate evolution strategy is employed, allowing cautious early training followed by refined parameter updates in later epochs.

The final prediction, along with its confidence score, is forwarded to the **decision support layer**, where ripeness classifications are translated into actionable storage and handling recommendations. Unripe fruits are advised for room-temperature storage to allow natural ripening, ripe fruits are recommended for immediate consumption or short-term refrigeration, and overripe fruits are suggested for refrigeration or processing.

The **web interface layer** completes the implementation by providing a human–system interaction platform. Users can upload images, view ripeness predictions, observe confidence levels, and receive storage advice in an intuitive and transparent manner. A feedback loop between the model and interface ensures that AI-generated outputs are communicated clearly, enhancing user trust and enabling informed decision-making.

Overall, the implementation integrates EfficientNetB0-driven visual reasoning, robust preprocessing, adaptive optimization, and user-centric presentation into a unified system capable of accurate, efficient, and practical fruit ripeness detection.

CHAPTER-5
SOFTWARE ENVIRONMENT

5. Software Environment

The software environment for the fruit ripeness detection system is designed to support efficient image processing, deep learning–based model training, evaluation, and deployment. The system leverages widely adopted open-source frameworks and libraries to ensure scalability, reproducibility, and ease of development. The core development and experimentation are carried out using the Python programming language, which serves as the primary platform due to its extensive ecosystem for machine learning and computer vision applications. Python provides seamless integration between data preprocessing, model training, and evaluation workflows.

For deep learning model development, TensorFlow with its high-level API Keras is used. TensorFlow enables the construction, training, and fine-tuning of convolutional neural networks, while Keras simplifies model design and experimentation. Transfer learning is implemented using pre-trained architectures such as EfficientNetB0, allowing the system to benefit from features learned on large-scale datasets like ImageNet. OpenCV is employed for image preprocessing tasks, including resizing, normalization, color space conversion, and enhancement operations. These preprocessing steps ensure consistency across input images and improve the quality of visual features extracted by the neural network. Additionally, data augmentation techniques such as rotation, flipping, zooming, and shifting are applied to increase dataset diversity and improve model generalization.

For data handling and numerical computation, NumPy and Pandas are utilized. NumPy enables efficient numerical operations on image arrays, while Pandas is used to manage dataset labels, image paths, and metadata in a structured format. These libraries simplify dataset organization and facilitate smooth integration with machine learning pipelines. Model evaluation and dataset partitioning are performed using Scikit-learn. This library is used to split the dataset into training, validation, and testing sets, ensuring unbiased performance evaluation. It also provides essential evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis, which are critical for assessing class-wise ripeness prediction performance.

The model training process is enhanced through the use of Keras callbacks, including model checkpointing, early stopping, and learning rate schedulers. These tools help prevent overfitting, retain the best-performing model, and improve convergence stability during training.For experimentation, visualization, and result analysis, Jupyter Notebook or Google Colab is used as the development environment. These platforms provide interactive execution, GPU acceleration, and easy visualization of training curves, confusion matrices, and classification reports.

CHAPTER-6
SYSTEM TESTING

6. System Testing

System testing is a crucial phase in the development of the fruit ripeness detection system, as it ensures that all integrated software components function correctly and that the system meets its intended performance, accuracy, and usability requirements. The primary goal of system testing is to validate the complete workflow—from image input to ripeness prediction and output visualization—under real-world conditions.

The testing process begins with functional testing of the image input module. This phase verifies that the system correctly accepts fruit images of varying resolutions, formats (JPEG, PNG), lighting conditions, and backgrounds. The system is tested with images captured from different sources, including mobile phones and digital cameras, to confirm robustness against real-world image variability. Successful image upload and preprocessing confirmation indicate correct functioning of this module.

Next, preprocessing validation testing is conducted to ensure that input images are resized, normalized, and formatted correctly before being passed to the EfficientNetB0 model. Test cases include images with inconsistent dimensions, color variations, and noise. The output of the preprocessing stage is verified to match the expected input shape and data distribution required by the trained neural network.

Model performance testing forms the core of system testing. The trained EfficientNetB0-based classification model is evaluated using a held-out test dataset derived from the Kaggle Fruit Ripeness dataset. Standard evaluation metrics such as accuracy, precision, recall, and F1-score are computed to assess overall and class-wise performance. A confusion matrix is generated to identify misclassification patterns among Unripe, Ripe, and Overripe classes, helping to detect potential confusion between visually similar ripeness stages.

To address dataset imbalance and real-world variability, generalization testing is performed using unseen images not included in the training dataset. These tests assess the system's ability to correctly classify fruits under different lighting, orientation, and surface texture conditions. Consistent prediction accuracy across these variations indicates strong generalization capability.

Stress and reliability testing is carried out by repeatedly submitting multiple image inputs to the system to evaluate its stability and response consistency. The system is monitored for failures, incorrect predictions, or excessive response delays. The use of early stopping, model checkpointing, and optimized inference pipelines ensures stable and reliable performance during continuous operation.

CHAPTER-7
SAMPLE CODE

7. Sample code

requirements.txt

```
tensorflow>=2.8.0
numpy>=1.19.5
pandas>=1.3.5
matplotlib>=3.5.1
seaborn>=0.11.2
scikit-learn>=1.0.2
Pillow>=9.0.0
streamlit>=1.8.1
opencv-python-headless>=4.5.5.64
albumentations>=1.1.0
scikit-image>=0.19.2
ipykernel>=6.9.1
jupyter>=1.0.0
black>=22.1.0
flake8>=4.0.1
pytest>=7.0.1
googletrans==3.1.0a0
```

data_preprocessing.py

```
CLASS DataPreprocessor:
    METHOD __init__(data_dir, img_size, batch_size, test_size, val_size):
        SET data directory, image size, batch size, test size, validation size
        INITIALIZE variables for class indices and class names
        CREATE training data generator with augmentations and validation split
        CREATE test data generator without augmentations
    METHOD get_data_generators():
```

```
    LOAD training data from "Train" folder using training generator (subset=training)
    STORE class indices and class names
    SAVE class indices to JSON file
    LOAD validation data from "Train" folder using training generator (subset=validation)
    LOAD test data from "Test" folder using test generator
    CALCULATE class weights based on training data distribution
    RETURN train_generator, val_generator, test_generator, class_weights
METHOD _calculate_class_weights(generator):
    COUNT number of samples per class
    COMPUTE weights inversely proportional to class frequency
    RETURN dictionary of class weights
METHOD _save_class_indices(generator):
    IF generator has class indices:
        INVERT mapping (index → class name)
        CREATE "models" folder if not exists
        SAVE mapping to "models/class_indices.json"
METHOD plot_sample_images(generator, num_images):
    GET one batch of images and labels from generator
    FOR each image in batch (up to num_images):
        DISPLAY image with predicted class name
    SAVE plot to "logs/sample_images.png"
METHOD get_class_distribution(data_dir):
    IF no data_dir provided, use "Train" folder
    FOR each class folder in data_dir:
        COUNT number of image files (.png, .jpg, .jpeg)
        STORE counts in dictionary
    RETURN class_counts
METHOD plot_class_distribution(class_counts, save_path):
    IF no class_counts provided, call get_class_distribution()
    IF class_counts is empty, print "No classes found"
    SORT classes by count (descending)
```

PLOT bar chart of class distribution
LABEL bars with counts
SAVE plot if save_path provided
SHOW plot
RETURN class_counts

MAIN PROGRAM:
CREATE DataPreprocessor instance
PRINT "Analyzing dataset..."
CALL plot_class_distribution() and save plot to "logs/class_distribution.png"
PRINT class distribution counts
PRINT "Preparing data generators..."
CALL get_data_generators() → train_gen, val_gen, test_gen, class_weights
PRINT class indices and class weights
GET one batch from train_gen
PRINT batch shape, labels shape, number of classes
CALL plot_sample_images(train_gen)
PRINT "Sample images saved"

model.py

CLASS RipenessModel:
METHOD __init__(num_classes, img_size, learning_rate):
SET number of classes, image size, learning rate
SET total epochs = 20
BUILD model using _build_model()
METHOD _build_model():
LOAD EfficientNetB0 base model (pretrained on ImageNet, no top layers)
FREEZE base model layers
DEFINE input layer with given image size
PREPROCESS input for EfficientNet
PASS input through base model
APPLY global average pooling

```
    ADD dense layer (1024 units, swish activation)
    ADD batch normalization
    ADD dropout (0.3)
    ADD dense layer (512 units, swish activation)
    ADD batch normalization
    ADD dropout (0.2)
    ADD squeeze-excitation block:
        dense layer (32 units, swish activation)
        dense layer (512 units, sigmoid activation)
        multiply with previous features
    ADD final dense layer (num_classes units, softmax activation, L2 regularization)
    RETURN complete model
METHOD compile_model():
    DEFINE categorical crossentropy loss with label smoothing
    DEFINE AdamW optimizer with learning rate, weight decay, etc.
    COMPILE model with optimizer, loss, and metrics:
        accuracy, precision, recall, AUC, top-2 accuracy
METHOD get_callbacks(log_dir, checkpoint_path):
    CREATE directories for logs and checkpoints
    DEFINE checkpoint callback (save best weights by val_accuracy)
    DEFINE early stopping callback (monitor val_loss, patience=10)
    DEFINE learning rate scheduler (warmup first 5 epochs, cosine decay after)
    DEFINE reduce learning rate on plateau callback
    DEFINE tensorboard callback for logging
    DEFINE CSV logger callback
    RETURN list of callbacks
METHOD unfreeze_layers(num_layers):
    SET model trainable = True
    FREEZE all but last num_layers of base model
    GET current learning rate
    RECOMPILE model with Adam optimizer at current learning rate
```

PRINT confirmation of unfrozen layers and learning rate

METHOD save_model(model_dir):

CREATE directory if not exists

SAVE model architecture to JSON file

SAVE model weights to H5 file

PRINT confirmation of save

METHOD load_weights(weights_path):

LOAD weights into model

PRINT confirmation of load

METHOD set_total_epochs(total_epochs):

UPDATE total_epochs

PRINT confirmation of new total epochs

MAIN PROGRAM:

CREATE RipenessModel instance

COMPILE model

PRINT model summary

train.py

SET random seeds for reproducibility

DEFINE CONFIG dictionary:

- image size
- batch size
- initial training epochs
- fine-tuning epochs
- learning rates (initial and fine-tune)
- number of classes
- data directory
- model directory
- logs directory
- test/validation split sizes

FUNCTION plot_training_history(history, fine_tune_history):

```
    IF fine_tune_history exists:
        MERGE metrics from fine_tune_history into history
    PLOT training vs validation accuracy across epochs
    PLOT training vs validation loss across epochs
    SAVE plots to logs directory
FUNCTION plot_confusion_matrix(y_true, y_pred, class_names):
    COMPUTE confusion matrix
    PLOT heatmap with true vs predicted labels
    SAVE plot to logs directory
FUNCTION evaluate_model(model, test_generator, class_names):
    PRINT "Evaluating on test set"
    EVALUATE model on test data → test metrics
    GENERATE predictions on test data
    CONVERT predictions to class indices
    GET true labels from test generator
    PRINT classification report
    SAVE classification report to JSON file
    CALL plot_confusion_matrix(y_true, y_pred, class_names)
    RETURN test metrics
FUNCTION main():
    CREATE model and logs directories if not exist
    PRINT "Initializing data preprocessor"
    INITIALIZE DataPreprocessor with config parameters
    PRINT "Loading data"
    GET train, validation, test generators and class weights
    GET class names from preprocessor or generator
    PRINT "Initializing model"
    CREATE RipenessModel with num_classes and image size
    COMPILE model
    SET total epochs = initial + fine-tune epochs
    CONFIGURE callbacks (checkpoint, early stopping, LR scheduler, etc.)
```

PRINT "Starting initial training"
TRAIN model for initial_epochs with train/validation data and callbacks
PRINT "Starting fine-tuning"
UNFREEZE last 20 layers of base model
SET fine-tuning learning rate
TRAIN model for fine_tune_epochs starting from last epoch
PLOT training history (initial + fine-tune)
SAVE model architecture and weights
CALL evaluate_model(model, test_generator, class_names)
PRINT "Training completed successfully"
PRINT model and logs directory paths
IF script is run directly:
CALL main()

app.py

INITIALIZATION

Define supported FRUITS (Apple, Banana, Mango, Orange, Tomato).
Define RIPENESS_LEVELS (Unripe, Ripe, Overripe).
Create a STORAGE_TIPS dictionary mapped to ripeness levels.
Initialize translation engine and set default language to English.
Setup UI styling (Dark Mode CSS).

FUNCTION: Load_Model()

Check if model weights file exists; if not, exit with error.
Initialize EfficientNetB0 as the base architecture.
Build custom top layers:
Global Average Pooling.
Dense layers with Swish activation and Batch Normalization.
Squeeze-and-Excitation (SE) block for feature recalibration.
Final Softmax layer (12 classes: 4 fruits $\times$ 3 ripeness states).
Load pre-trained weights and compile with AdamW optimizer.

FUNCTION: Process_Image(input_image)

Convert image to RGB.

Resize image to 224×224 pixels.

Convert to Numerical Array.

Apply EfficientNet-specific preprocessing (scaling/normalization).

Return processed tensor.

FUNCTION: Translate(text, target_language)

If target is English, return original text.

Check if translation exists in Session Cache.

If not, call Translation API and store result in cache.

Return translated text.

MAIN APP LOOP

Sidebar:

Render language selection dropdown.

If language changes, clear cache and refresh UI.

Render toggle between "Camera" and "File Upload".

Input Stage:

Capture image via Camera or File Uploader.

Analysis Stage (On "Analyze" Button Click):

Preprocess image using Process_Image().

Pass image to Model.predict().

Calculate Indices:

Ripeness_Index = Highest_Probability_Index MOD 3

Fruit_Index = Highest_Probability_Index DIV 3

Generate label (e.g., "a_ripe" for Apple-Ripe).

Overlay Graphics: Draw a bounding box and label on the image.

Display Results:

Show annotated image.

Display Ripeness State with color coding (Red for unripe, Green for ripe).

Display Confidence Score percentage.

Expandable section: Show Storage Tips based on detected ripeness.

Performance Metrics (Footer):

Display static model metrics (Accuracy, Precision, F1-Score).

Render a bar chart of class-wise performance using Plotly.

Render an HTML table for detailed metric breakdown.

\

CHAPTER-9
RESULTS

8. Results

At train.py

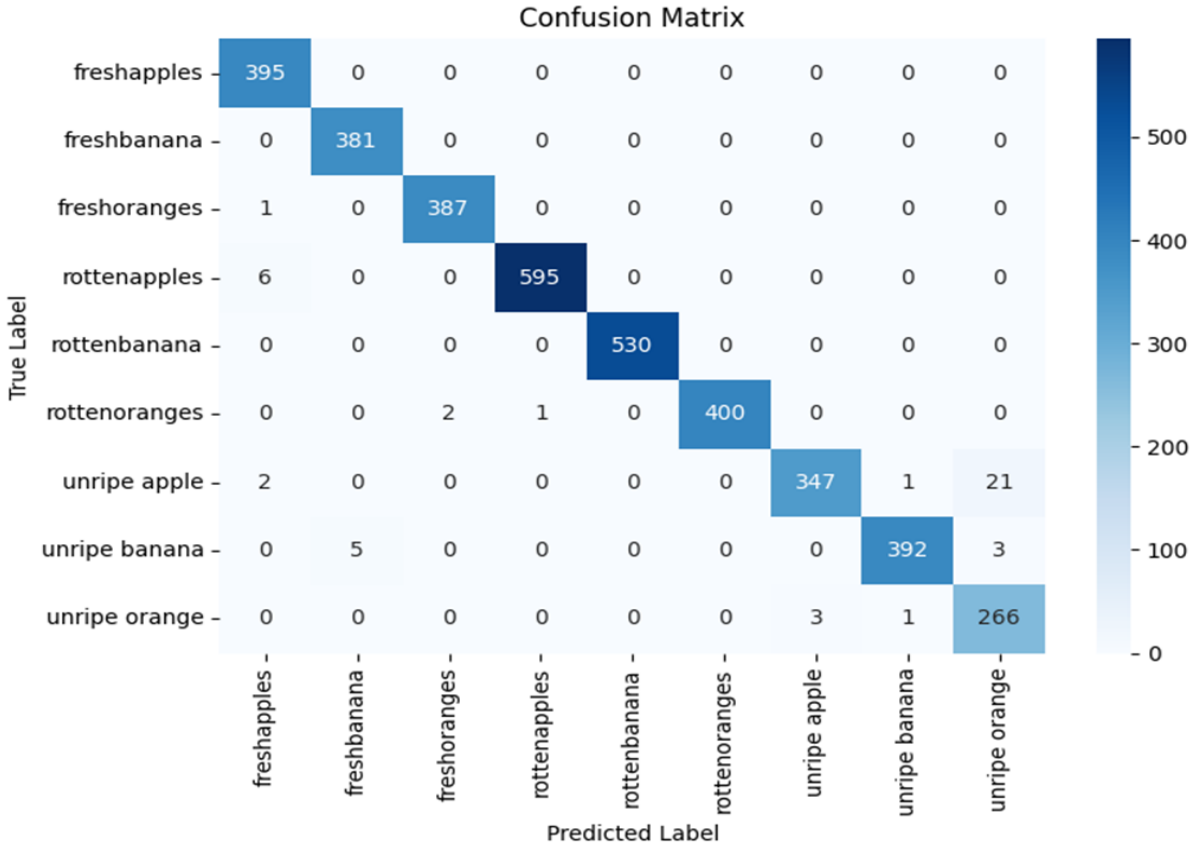

Fig:9.1 Confusion Matrix and Class-wise Precision-Recall Curve for Proposed EfficientNetB0 for Ripeness Classification.

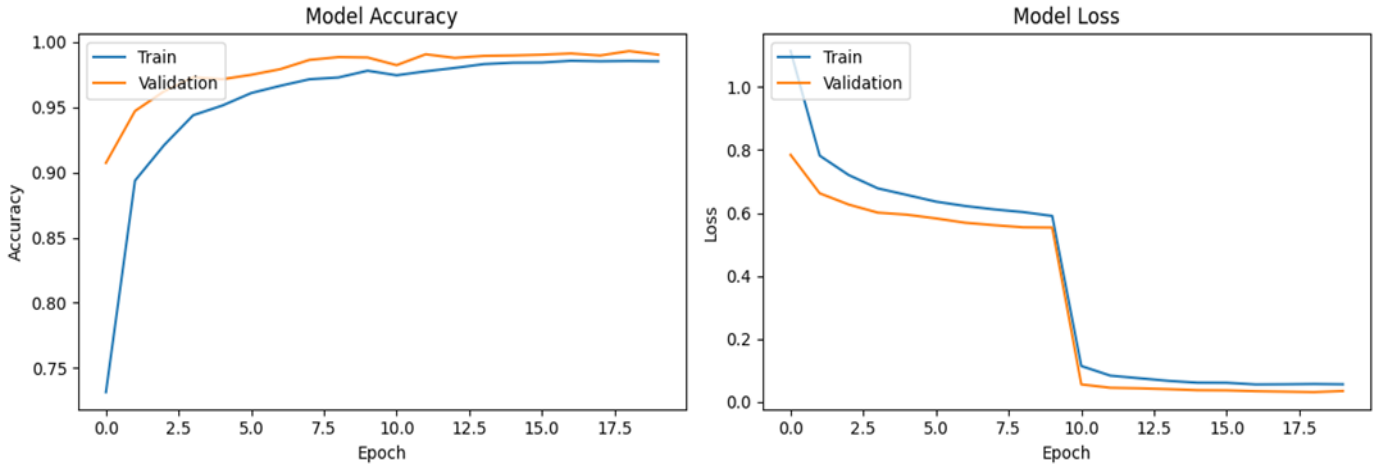

Fig:9.2 Accuracy and loss curves of the EfficientNetB0: Train and Validation sets.

At data_preprocessing.py

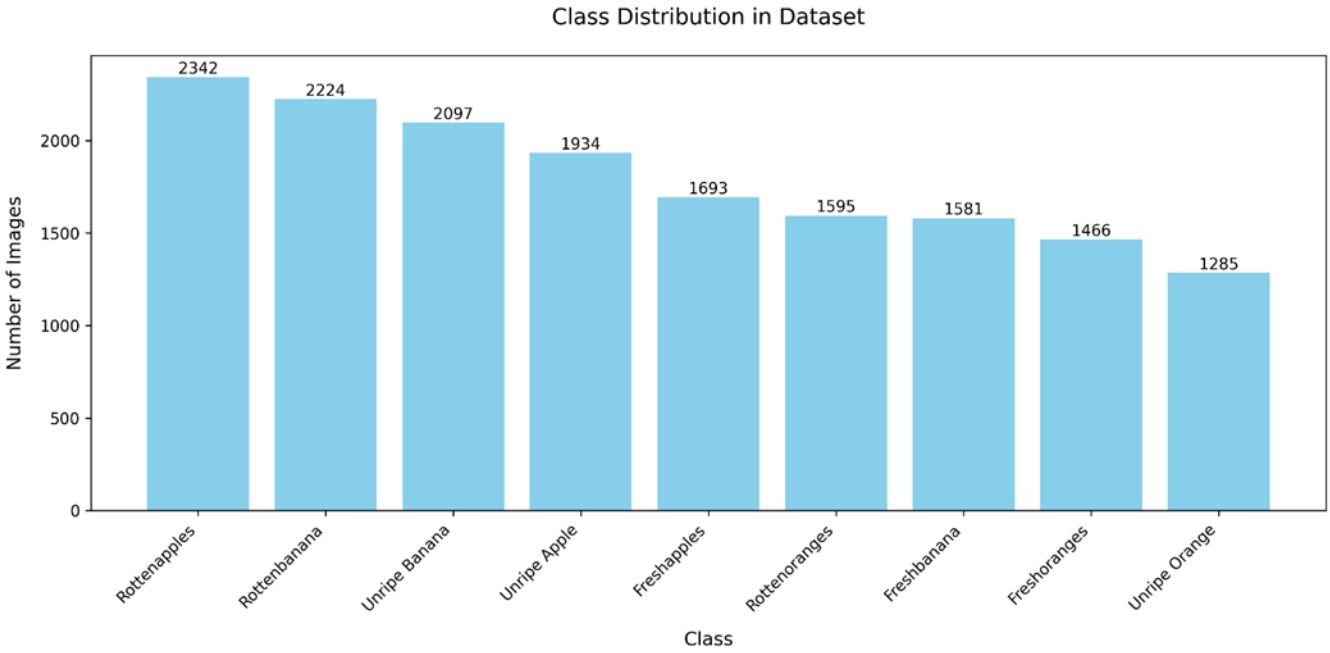

Fig:9.3 Class distribution of the dataset which is classified based on no.of images for different class.

Fig:9.4 Sample Images while training the model.

At app.py

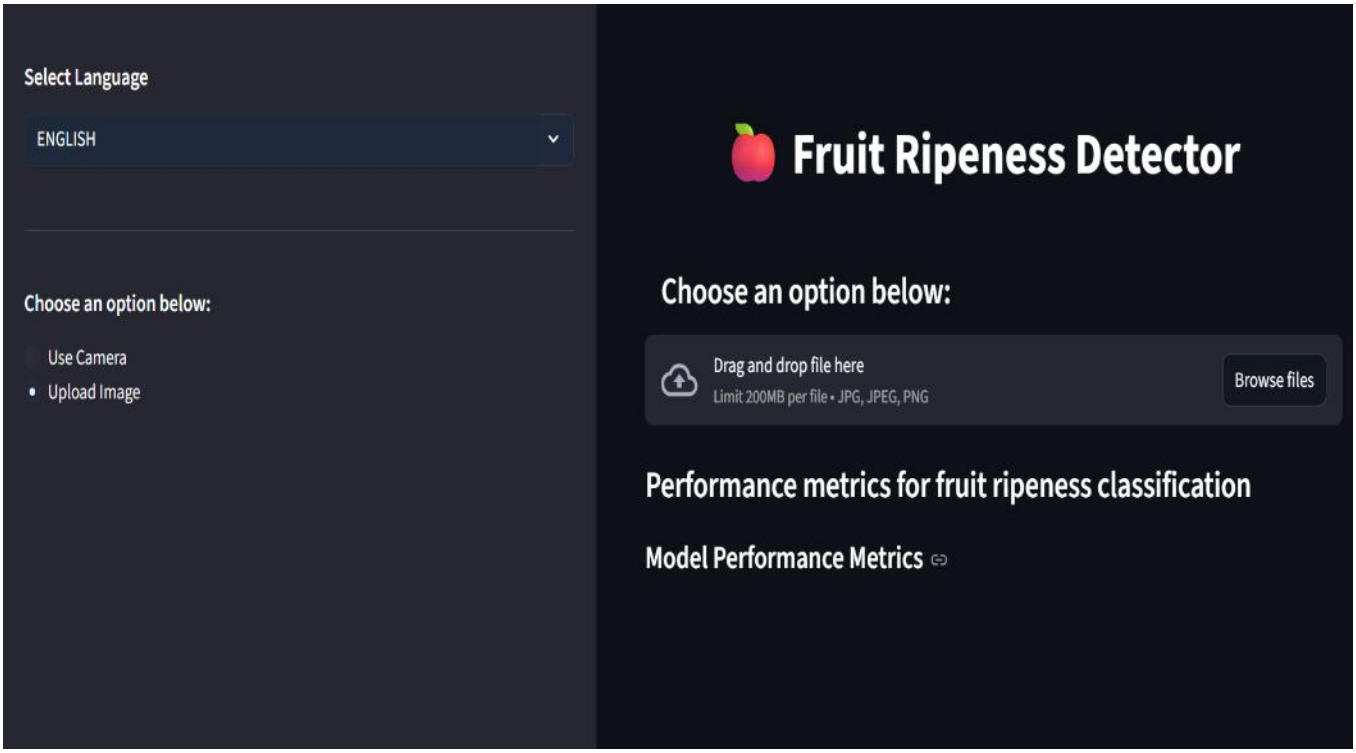

Fig:9.5 The interface first look of Fruit Ripeness Detector.

Fig:9.6 The fruit taken for detecting the ripeness.

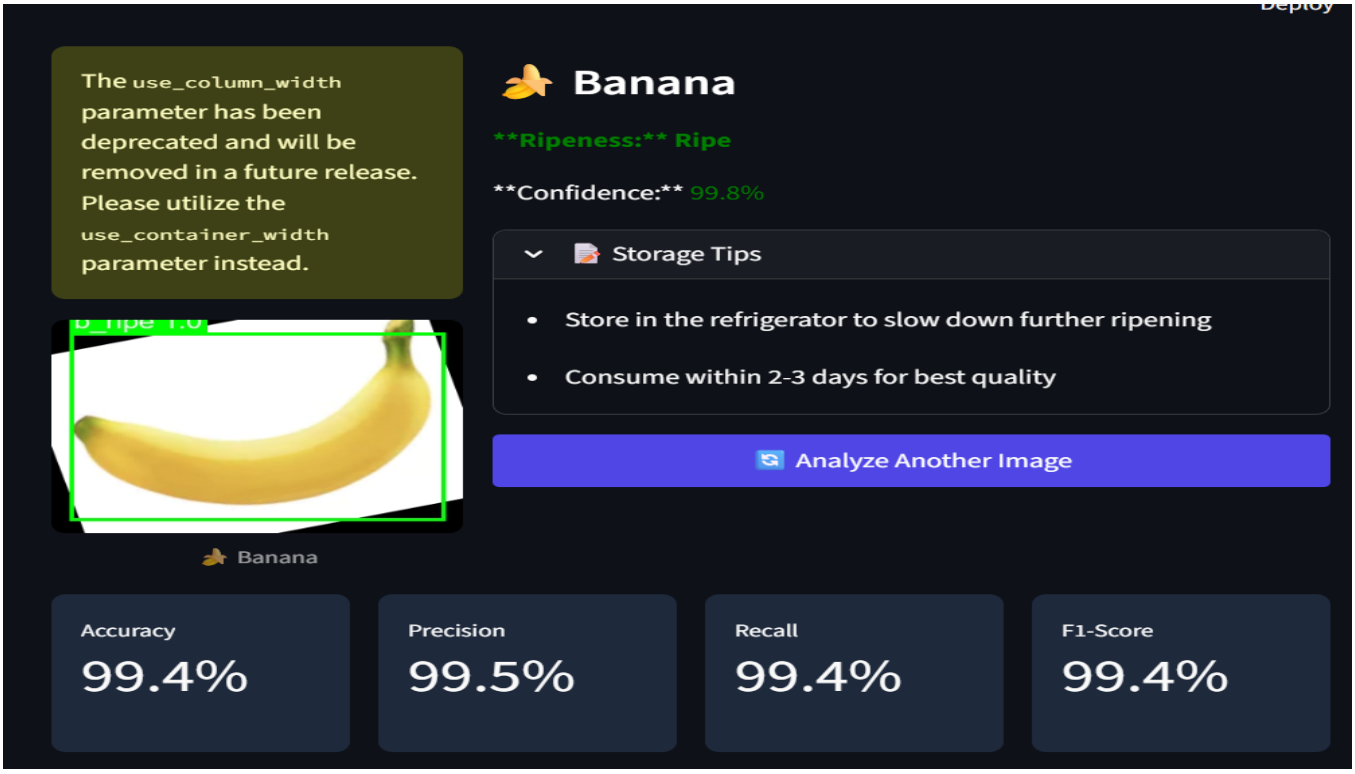

Fig:9.7 After analyzing the fruit it shows the Ripeness class, Confidence, Storage tips.

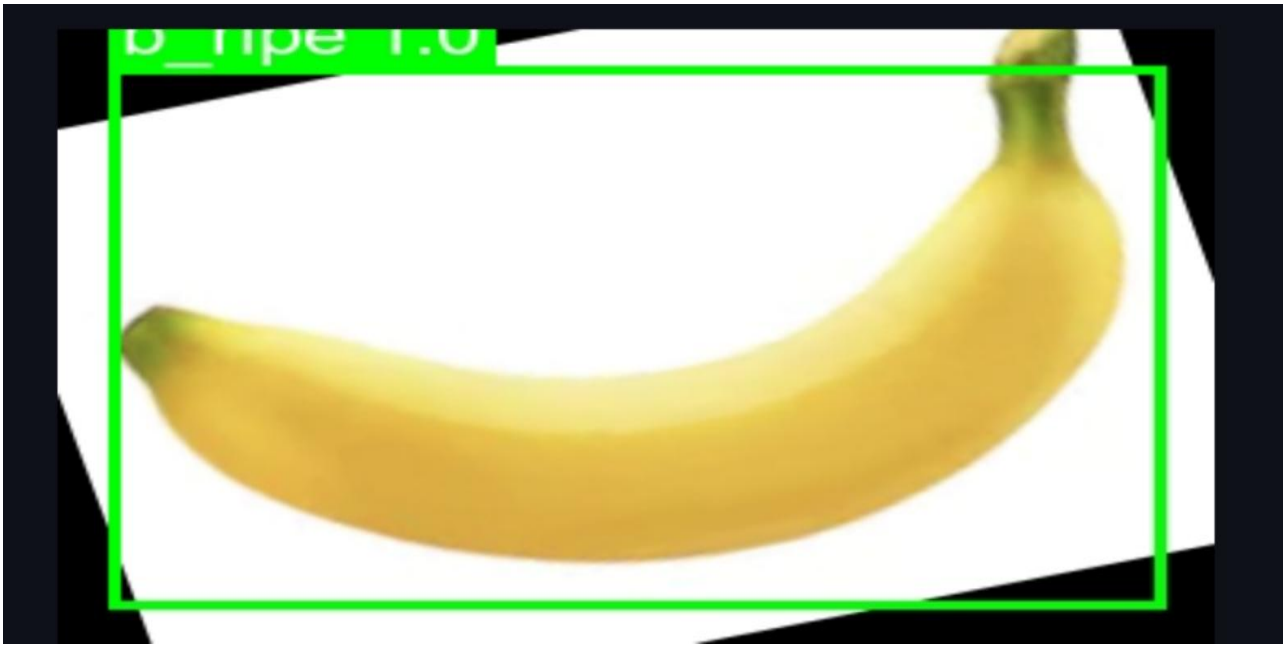

Fig:9.8 Fruit is having the tagging along with the ripeness stage.

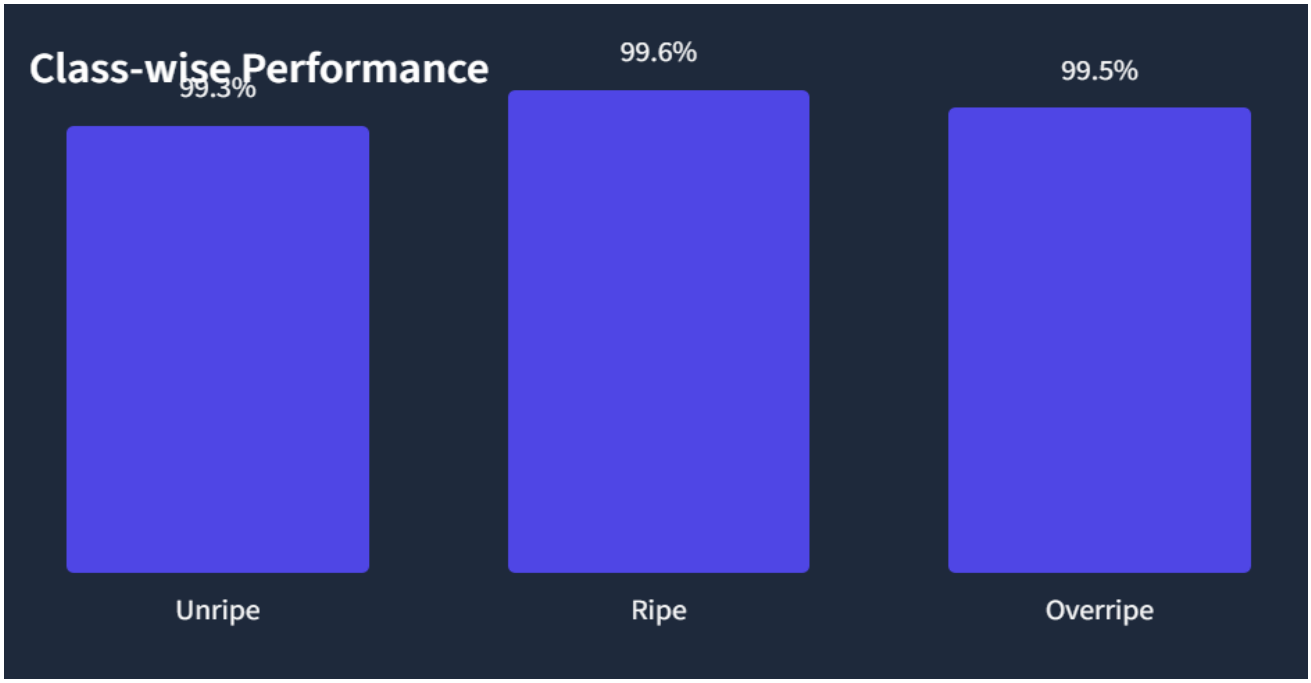

Fig:9.9 Class Distribution for the fruit detection in a graph.

Class-wise Performance

Class	Precision	Recall	F1-Score
Unripe	99.3%	99.4%	99.3%
Ripe	99.6%	99.4%	99.5%
Overripe	99.5%	99.4%	99.5%

Fig:9.10 Performance metrics percentage of the fruit.

CHAPTER-10
CONCLUSION

9. Conclusion

Conclusion

In this article, I have discussed the intelligent fruit ripeness inspection system designed for effective classification of the ripeness level of multiple fruits using deep learning. An CNN model has been proposed for the classification of fruits at three levels of ripeness, namely Unripe, Ripe, and Overripe, and achieved a classification accuracy of 99.4%. Apart from fruit ripeness analysis, the proposed system also provides specific advice on fruit storage for each ripeness level, assisting users in taking a final decision on whether the fruits need to be stored in the fridge, be consumed immediately, or be stored at room temperature. Data augmentation has been employed to counter the issue of limited training images and improve the robustness of the proposed model for fruit classification. A user-friendly web interface has also been developed for multilingual online prediction services.

The findings of this study have confirmed that the technology of computer vision and deep learning is capable of successfully automating the ripeness prediction task and thus can serve as a reliable and non-destructive means of quality evaluation, offering a more efficient and objective approach to quality assessment for manual evaluation techniques. The technology has the capacity to decrease the amount of agricultural losses by up to 15-20% and is thus, highly capable of offering a reliable means of quality assessment and value addition for agricultural producers and thus potentially applicable for quality assessment within agricultural value chains. The proposed technology is applicable for quality assessment and decision-making within agricultural value chains and capable of offering more value and efficiency within agricultural value chains due to its efficiency and objectiveness. The proposed system thus tackles major limitations and challenges of agricultural technology and agricultural value chain decision-making and capable of application within small and large agricultural ventures for quality assessment and decision support. The proposed system and technology thus capable of application for agricultural quality assessment and decision support and thus potentially capable of offering major economic and ecological benefits through successful application for agricultural decision support within agricultural value chains.

CHAPTER-11
REFERENCES

10. References

[1] .P. Kamat, S. Gite, H. Chandekar, L. Dlima, and B. Pradhan, "Multi-class fruit ripeness detection using YOLO and SSD object detection models," Discover Applied Sciences, vol. 7, 2025.

[1] A. Authors et al., "Deployment of an IoT and ML-driven platform for fruit ripeness evaluation and spoilage detection: A case study on bananas," Journal of Food Quality, 2024.

[2] H. Yu et al., "Ripe-detection: AI-lightweight method for strawberry ripeness detection," Agronomy, vol. 15, no. 7, p. 1645, 2025.

[3] A. Authors et al., "Deployment of CES-YOLO: An optimized YOLO-based model for blueberry ripeness detection on edge devices," Scientific Reports, vol. 15, 2025.

[4] A. Authors et al., "Hybrid attention transformer integrated YOLOv8 for fruit ripeness detection," ScienceDirect, 2025.

[5] A. Authors et al., "A banana ripeness detection model based on improved YOLOv9c in multifactor complex scenarios," Agronomy, vol. 15, no. 5, p. 1173, 2025.

[6] İ. Ünal et al., "A lightweight instance segmentation model for simultaneous detection of citrus fruit ripeness and red scale (Aonidiella aurantii) pest damage," Applied Sciences, vol. 15, no. 17, p. 9742, 2025.

[7] A. Authors et al., "A lightweight citrus ripeness detection algorithm based on visual saliency priors and improved RT-DETR," Agronomy, vol. 15, no. 8, p. 1948, 2025.

[8] A. Authors et al., "Improved RT-DETR and its application to fruit ripeness detection," ProQuest Dissertations & Theses, 2025.

[9] J. Y. Goh, Y. M. Yunos, and M. S. M. Ali, "Fresh fruit bunch ripeness classification methods: A review," Food and Bioprocess Technology, 2024.

[10] **Salim & Mohammed (2024)**: Systematic review of machine learning and deep learning methodologies applied to fruit ripeness detection.

[11] Zhou et al. (2025): Comparative efficacy of deep learning methodologies in vision-based systems for efficient and accurate assessment in agricultural automation.

[12] **Zhang (2023)**: Evaluation of deep learning approaches for superior accuracy in ripeness detection and challenges in complex scenarios such as variable lighting and occlusions.

[13] Goh et al. (2024): Analysis of the scalability and end-to-end learning capabilities of CNNs to eliminate manual preprocessing.

[14] Vo et al. (2024): Improved DenseNet architectures for managing dataset imbalances and sparse data in real-world agricultural settings.

[15] Wang et al. (2025): Suitability of YOLO variants for edge devices and field deployment in precision agriculture.

[16] Xiao et al. (2023): Handling multi-class ripeness stages effectively using advanced object detection models.

[17] Singh et al. (2025): Implementation of CNN-based lightweight models for low computational cost on mobile and IoT platforms.

[18] **A. Authors et al. (2025)**: Deployment of CES-YOLO: An optimized YOLO-based model for blueberry ripeness detection on edge devices (Scientific Reports).

[19] **H. Yu et al. (2025)**: Ripe-detection: AI-lightweight method for strawberry ripeness detection (Agronomy).

[20] **A. Authors et al. (2025)**: Hybrid attention transformer integrated YOLOv8 for fruit ripeness detection (ScienceDirect).

[21] **A. Authors et al. (2025)**: A banana ripeness detection model based on improved YOLOv9c in multifactor complex scenarios (Agronomy).

[22] **İ. Ünal et al. (2025)**: A lightweight instance segmentation model for simultaneous detection of citrus fruit ripeness and pest damage (Applied Sciences).

[23] **A. Authors et al. (2025)**: A lightweight citrus ripeness detection algorithm based on visual saliency priors and improved RT-DETR.

[24] **A. Authors et al. (2024)**: Deployment of an IoT and ML-driven platform for fruit ripeness evaluation and spoilage detection (Journal of Food Quality).

[25] **YOLO (v5, v6, v7, v8, v9c)**: Utilized for real-time detection, single-stage object detection, and high accuracy with low inference time.

[26] **EfficientNetB0**: The primary backbone used in this project for high accuracy with reduced computational cost.

[27] **MobileNetV2**: A lightweight CNN backbone designed for mobile and embedded systems, achieving 99.3% accuracy in previous methods.

[28] **Transformer-based Models (RT-DETR, Vision Transformer)**: Emerging trends for enhanced global context understanding and accuracy in complex backgrounds.

[29] **SSD-MobileNetV1/V2**: Employed for its computational efficiency and suitability for edge-device deployment.

PUBLICATION STATUS